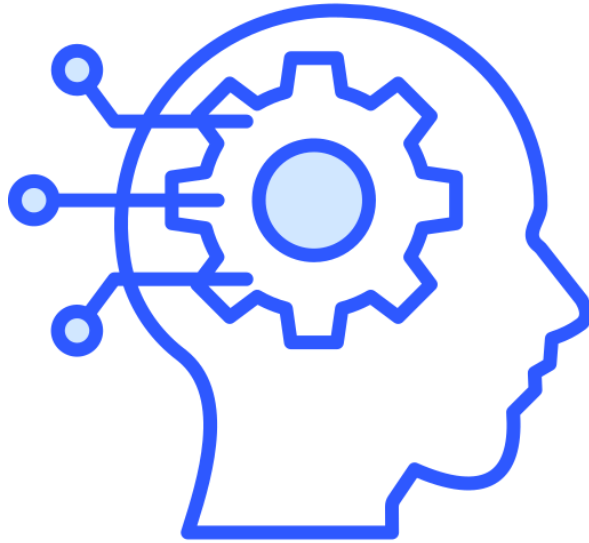


HEURISTICS AND METAHEURISTICS FOR OPTIMIZATION AND LEARNING

Prof. Mario Francesco Pavone
AA 2024/25



REALIZZATO DA
Giulio Pedicone

REALIZZATO DA
Vincenzo Villanova

REALIZZATO DA
Francesco Virzì



Indice

Introduzione	5
Le decisioni nella vita quotidiana	5
Cosa significa prendere una decisione?	6
Il test di Turing	6
Visione Locale vs Visione Globale.....	7
Algoritmi deterministici: come funzionano?	8
Ottimizzazione e Complessità	9
Esempi di problemi decisionali.....	9
Metodologie per risolvere problemi complessi	10
Iterativi vs Greedy e Stocastici vs Deterministici	11
A cosa fare attenzione nella risoluzione dei problemi	11
Chiave del successo: Concetti	11
Quando un problema è complesso?	12
Spazio di Ricerca.....	12
Chiavi di successo di un algoritmo	13
Il problema del Feedback Vertex Set (o Feedback Arc Set).....	13
La funzione obiettivo	14
Funzione obiettivo e spazio di ricerca	14
Problemi di ottimizzazione con vincoli.....	14
Penalizzazione.....	15
Spazio di ricerca vs Spazio delle soluzioni	15
Panorama combinatoriale (Search Landscape).....	16
Teoria della complessità.....	17
Cos'è un problema.....	17
Esempio: il problema del commesso viaggiatore (TSP)	17
Landscape di un problema	18
Esempio: problema K-SAT	18
Algoritmi.....	18
Perché conoscere la teoria della NP-completezza.....	19
Problema: Punto iniziale	19
Le classi P e NP	21

Classe di complessità P	21
Classe NP	21
Il problema del ciclo hamiltoniano	22
Problemi NP-completi.....	23
Riducibilità.....	23
NP-HARD ed NP-COMPLETE	24
Teorema.....	24
SAT (Satisfiability).....	25
Teorema di Cook-Levin.....	26
Riduzione	26
Traveling Salesman Problem	27
Clique	28
3SAT riducibile in tempo polinomiale a CLIQUE.....	29
Vertex Cover.....	30
Ottimi Locali vs Ottimi Globali	31
Cos'è l'ottimizzazione vincolata?	32
Tecniche di Gestione dei Vincoli	32
Approccio delle Funzioni di Penalità.....	33
Funzioni di Penalità Statiche	34
Funzioni di Penalità Dinamiche	34
Funzioni di Penalità Adattive.....	36
Funzione di Fitness e Selezione	36
Algoritmo di Riparazione	37
Algoritmo di Riparazione: Problemi implementativi.....	37
Problemi complessi	38
Ottimo Locale.....	38
Come esplorare lo spazio di ricerca (chiave del successo)	38
Vincoli temporali e quantità di informazioni	38
Ispirazione dalla natura	38
Teoria dell'evoluzione	39
Algoritmi Bio & Natural Inspired	39
Swarm Intelligence (Intelligenza dello sciame).....	39
Colonie di formiche	40
Funzionamento dei sistemi evolutivi	40
Perché sviluppare una nuova classe di algoritmi?	41

Confronti.....	41
Vantaggi.....	42
Metodi risolutivi dei problemi	43
Punti Chiave	43
Algoritmi Greedy.....	44
Algoritmi Greedy - Limitazioni.....	44
Algoritmi Iterativi	45
Ottimo Locale – Definizione	46
Soluzione Iniziale.....	46
Definizione di Vicinato.....	47
Tabu Search	51
Obiettivo.....	51
Introduzione alla Tabu Search	51
Funzionamento Base	52
Pseudocodice	53
Problemi di progettazione specifici per una Tabu Search semplice.....	53
Meccanismi avanzati comuni nella Tabu Search	53
Tabu Search applicato al Sudoku	54
Local Search	57
Algoritmo di Ricerca Locale	57
Pseudocodice	58
Selezione del Vicinato.....	58
Iterated Local Search.....	60
Pseudocodice	62
Metodi di perturbazione.....	63
Criteri di Accettazione.....	63
Perché un problema è complesso?	64
Algoritmi basati sulla popolazione	64
Costruire meccanismi ispirati alla natura	64
Teoria dell'evoluzione	65
Quattro concetti chiave.....	65
Tipologie di algoritmi ispirati alla natura	65
Swarm Intelligence	65
Colonie di formiche (Ant Colony Optimization).....	66

Evoluzione naturale	66
Algoritmo genetico	67
Pseudocodice	67
Come funziona	68
Operatori dell'algoritmo genetico	68
Exploitation ed Exploration	69
Operatore di Selezione	70
Selezione degli Individui per la Generazione Successiva	71
Crossover	72
Mutazione.....	74
Esempio di Applicazione di un Algoritmo Evolutivo.....	76
Ant Colony Optimization.....	78
Dalle formiche reali a quelle artificiali	78
Problemi di Ottimizzazione Combinatoria.....	80
Algoritmi di Swarm Intelligence	82
Ant Colony Algorithms.....	83
Ant System	84
Min-Max Ant System	85
Ant Colony System.....	86
Algoritmo ACO	87
Vantaggi e Svantaggi.....	88
Performance	88

Introduzione

Viviamo in un'epoca in cui l'intelligenza artificiale (IA) sta automatizzando molti aspetti della nostra vita quotidiana. L'obiettivo dell'IA non è sostituire l'essere umano, ma emulare il suo modo di ragionare e agire all'interno di un sistema computazionale.

L'intelligenza artificiale si articola in diverse aree di studio, tra cui:

- **Machine Learning (ML)**
- **Deep Learning (DL)**
- **Natural Language Processing (NLP)**

In un mondo scandito dal tempo, ottimizzare le prestazioni degli algoritmi è fondamentale: vogliamo che vengano eseguiti nel minor tempo possibile. Inoltre, viviamo nell'era dei **Big Data**, dove siamo sommersi da enormi quantità di dati da elaborare e da cui estrarre conoscenza utile — tutto ciò in tempi spesso molto ristretti.

Un esempio concreto: Google Maps non utilizza l'algoritmo di Dijkstra nella sua forma classica perché non è praticabile su dataset di dimensioni così elevate. Analizzare ogni possibile percorso richiederebbe troppo tempo. Questo evidenzia come gli **algoritmi deterministici tradizionali** risultino spesso inadeguati per problemi complessi su larga scala, soprattutto in presenza di incertezza o assenza di informazioni iniziali — come nel caso di un labirinto in cui conosciamo solo il punto di ingresso.

Le decisioni nella vita quotidiana

Ogni giorno prendiamo decisioni, spesso in modo automatico e inconsapevole. Le nostre scelte dipendono dagli **obiettivi** che vogliamo raggiungere. Alcune decisioni sono semplici, altre più complesse e richiedono tempo e ragionamento. Tuttavia, tutte devono essere prese in modo **rapido ed efficiente**.

Cosa significa prendere una decisione?

Significa avere un insieme di opzioni tra cui scegliere quella che meglio ci avvicina al nostro obiettivo. Idealmente, vogliamo **massimizzare il beneficio e minimizzare lo sforzo**.

Tuttavia, il processo decisionale è spesso complicato da diversi fattori:

- L'enorme quantità di informazioni disponibili
- I limiti di tempo
- L'incertezza sulle conseguenze delle azioni
- L'ambiente dinamico in continuo cambiamento

Per affrontare queste sfide, è necessario **agire in modo razionale**. Un sistema razionale è in grado di prendere la decisione migliore possibile, dato ciò che conosce. Tuttavia, gli algoritmi deterministici richiedono informazioni di partenza per funzionare: se queste mancano, non sono applicabili.

Ad esempio, non possiamo applicare l'algoritmo di Dijkstra in un labirinto se non conosciamo la struttura del percorso. Allo stesso modo, è impraticabile utilizzarlo su un grafo con milioni di nodi e archi, perché il tempo di calcolo diventerebbe proibitivo.

Il test di Turing

Il **Test di Turing** è un criterio proposto per determinare se una macchina possa essere considerata intelligente. Il test prevede due stanze separate: in una si trova un essere umano, nell'altra un interlocutore (che può essere un altro umano o una macchina). Se il primo non riesce a distinguere se sta interagendo con una persona o con una macchina, allora quest'ultima ha superato il test.

Per superare il test, una macchina deve **comprendere e rispondere alle informazioni in modo simile a un essere umano**, dimostrando capacità come:

- Interpretazione del **linguaggio naturale (NLP)**
- Rappresentazione della conoscenza
- Apprendimento per adattarsi
- Ragionamento automatico
- Visione artificiale
- Manipolare oggetti e muoversi

Per **superare il Test di Turing**, dobbiamo sviluppare metodologie computazionali in grado di funzionare efficacemente anche in ambienti caratterizzati da forte incertezza! La macchina, quindi, deve **agire come se fosse "bendata"**, affrontando situazioni sconosciute senza informazioni complete. In questi contesti, gli algoritmi deterministici sono spesso inapplicabili, perché non possono costruire soluzioni a partire dal nulla

Visione Locale vs Visione Globale

Nella maggior parte dei problemi di ottimizzazione, la **visione locale** adottata dalle euristiche greedy tende a ridurre le prestazioni rispetto agli algoritmi iterativi o globali, che invece sfruttano una **visione più ampia e strategica** del problema.

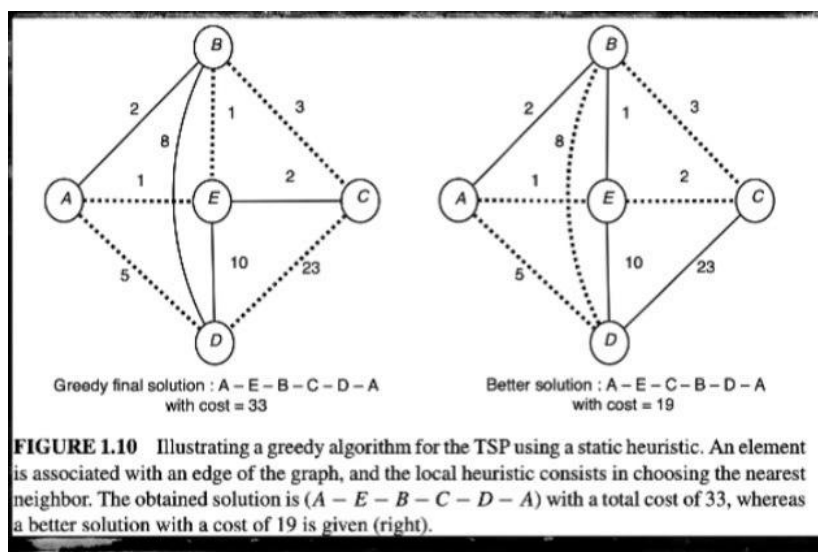
Cosa si intende per visione locale e visione globale?

- **Visione locale**: significa prendere decisioni basandosi solo sull'informazione immediatamente disponibile nel punto corrente del processo. In pratica, si considera solo ciò che è "vicino" o rilevante nel breve termine, senza valutare l'impatto futuro o globale delle scelte. È tipica delle strategie greedy.
- **Visione globale**: implica valutare l'intero spazio delle soluzioni o comunque una porzione ampia di esso, tenendo conto delle conseguenze a lungo termine delle decisioni. Questo approccio consente di individuare soluzioni più complete e ottimali, anche se meno immediate.

Differenze nei risultati

- **Soluzione greedy (visione locale)**: sceglie sempre l'opzione che appare migliore nel momento presente, come ad esempio il nodo più vicino o il guadagno immediato maggiore. Questo porta spesso a **soluzioni subottimali**, perché le decisioni non tengono conto della struttura complessiva del problema.
- **Soluzione ottimale (visione globale)**: segue un percorso che può apparire meno vantaggioso all'inizio, ma che porta a risultati migliori nel lungo periodo. Questo è possibile solo analizzando il problema nel suo insieme.

Le **euristiche greedy**, facendo affidamento sulla **visione locale**, sono rapide ma spesso inefficaci nei problemi complessi. Al contrario, gli **algoritmi iterativi** e quelli basati su una **visione globale** (come A*, branch and bound, o algoritmi genetici) tendono a produrre risultati migliori, poiché considerano in modo più ampio il contesto e le conseguenze delle scelte.



Algoritmi deterministici: come funzionano?

Un algoritmo deterministico costruisce una soluzione passo dopo passo, partendo da uno stato iniziale noto. Prendiamo come esempio il **problema del commesso viaggiatore**: un venditore deve visitare un insieme di città partendo da una di esse, attraversandole tutte una sola volta e tornando al punto di partenza, minimizzando la somma dei costi (distanze) tra le città.

La costruzione della soluzione parte da:

- Una città iniziale
- La selezione, ad ogni passo, della città successiva con **il minor costo di collegamento**

Tuttavia, questa strategia **non garantisce la soluzione ottimale**. Percorsi che sembrano più convenienti localmente possono portare a una soluzione globale più costosa. Gli algoritmi deterministici, infatti, basano le loro decisioni su **informazioni locali**.

Per superare questi limiti, ci si ispira spesso alla **natura** e al modo in cui gli organismi viventi prendono decisioni e risolvono problemi complessi in ambienti sconosciuti, adattandosi senza bisogno di conoscenze pregresse.

Ottimizzazione e Complessità

Ogni giorno, ognuno di noi prende costantemente decisioni durante le proprie attività quotidiane. Queste decisioni devono spesso essere prese in modo rapido ed efficace, soprattutto quando ci troviamo di fronte a una grande quantità di informazioni. Di conseguenza, il processo decisionale è spesso complesso e difficile da affrontare. Prendere la decisione più razionale significa ottimizzare i risultati sulla base delle informazioni disponibili e dei vincoli presenti. Il processo decisionale razionale consiste nell'ottimizzare gli esiti tenendo conto dei vincoli dati. Prendere una decisione, in ambito computazionale, significa guidare un algoritmo verso la soluzione ottimale di un problema, selezionando la migliore tra un insieme di opzioni per raggiungere un obiettivo. Questo processo implica l'ottimizzazione di uno o più parametri. Quando si hanno molte informazioni, prendere decisioni è più semplice. Con poche informazioni, diventa una sfida più complessa.

Esempi di problemi decisionali

- **Illuminazione ottimale:** posizionare il minor numero di luci per illuminare uniformemente un edificio, ottimizzando la potenza utilizzata.
- **Evacuazione:** collocare un numero minimo di punti di uscita massimizzando l'accessibilità e minimizzando i percorsi.
- **Produzione industriale:** usare il minor numero di macchinari massimizzando produttività, guadagno e vendite.
- **Costruzione di immagini:** ridurre l'errore tra l'immagine ottenuta e quella desiderata, cercando la sequenza che minimizza tale errore.
- **Sport:** massimizzare il risultato (es. vittoria) minimizzando il tempo o l'energia impiegata per la preparazione.
- **Studio:** superare un esame impiegando il minor tempo possibile.

Esempio di Ottimizzazione: Gioco del 9 (8-puzzle)

Il problema del "gioco dell'8" consiste in una griglia 3×3 contenente otto tessere numerate da 1 a 8.

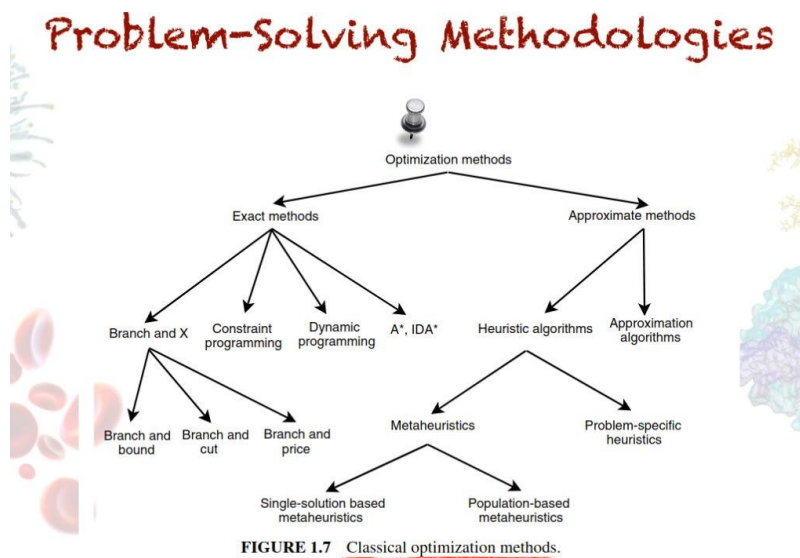
Uno dei riquadri della griglia (chiamato "blank") è vuoto.

Una tessera può essere spostata nello spazio vuoto da una posizione adiacente, creando così un nuovo spazio vuoto nella posizione originaria della tessera.

L'obiettivo è raggiungere una configurazione finale partendo da una posizione iniziale data, utilizzando il numero minimo di mosse.

L'insieme S per questo problema è l'insieme di tutte le sequenze di mosse che portano dalla configurazione iniziale a quella finale. La **funzione di costo f** di un elemento in S è definita come il numero di mosse nella sequenza.

Metodologie per risolvere problemi complessi



Algoritmi deterministici o esatti: non sempre adatti, soprattutto con spazi di ricerca molto grandi (es. matrici oltre il 3x3).

Due approcci principali:

1. **Metodi esatti**

- Basati su logica deterministica.
- Esplorano il problema in modo sistematico (es. Branch and Bound, A*).
- Eliminano i rami inutili dell'albero delle soluzioni.
- Funzionano bene finché lo spazio S è gestibile.

2. **Metodi approssimati**

- Si basano su componenti casuali o euristici.
- Partono da una o più soluzioni iniziali e cercano soluzioni migliori.
- Non garantiscono l'ottimo, ma trovano soluzioni molto vicine.
- Utili quando lo spazio è troppo grande per i metodi esatti.

Iterativi vs Greedy e Stocastici vs Deterministici

Le **metaeuristiche** possono essere classificate secondo due criteri principali: **deterministiche vs stocastiche** e **iterative vs greedy**.

Le **metaeuristiche deterministiche** seguono un processo decisionale fisso: partendo dalla stessa soluzione iniziale, si arriva sempre alla stessa soluzione finale (es. ricerca locale, tabu search). Al contrario, le **metaeuristiche stocastiche** introducono elementi di casualità nella ricerca (es. simulated annealing, algoritmi evolutivi), il che può portare a soluzioni finali diverse anche a partire dalla stessa condizione iniziale. Questo aspetto va considerato nella valutazione delle prestazioni. Dal punto di vista operativo, le **metaeuristiche iterative** partono da una soluzione (o popolazione di soluzioni) completa e la modificano iterativamente tramite operatori di ricerca. Gli **algoritmi greedy**, invece, costruiscono progressivamente la soluzione partendo da zero, aggiungendo una variabile alla volta. La maggior parte delle metaeuristiche appartiene alla categoria degli algoritmi iterativi.

A cosa fare attenzione nella risoluzione dei problemi

- La **complessità di un problema** fornisce un'indicazione sulla sua difficoltà.
- La **struttura delle istanze** gioca un ruolo importante.
- È fondamentale conoscere la **dimensione delle istanze di input**.
- Per alcuni problemi polinomiali molto grandi, o con **strutture particolari**, può essere necessario ricorrere all'uso di **metaeuristiche**.
- Il **tempo di ricerca richiesto** per risolvere il problema è un parametro importante: anche se il problema è polinomiale, l'uso di metaeuristiche può essere giustificato in presenza di **vincoli di tempo reale**.

Chiave del successo: Concetti

- **Codifica del problema**
- **Rappresentazione della soluzione**
- **Definizione della funzione obiettivo**
- **Progettazione del metodo/processo di ricerca della soluzione**

Quando un problema è complesso?

Un **problema è considerato** complesso quando:

- Lo spazio di ricerca è troppo grande per essere esplorato completamente.
- Non può essere risolto in tempo polinomiale (problemi NP-completi).

Esempi:

- **Problemi di classe P**: risolvibili in tempo polinomiale (es. Dijkstra, Spanning Tree).
- **Problemi di classe NP**: non risolvibili in tempo polinomiale (es. Knapsack Problem).

Nel problema dello zaino (Knapsack) bisogna selezionare oggetti per massimizzare il valore totale entro un certo peso. Anche se le informazioni sono note, l'alto numero di combinazioni rende difficoltoso l'utilizzo di metodi esatti su larga scala.

In ambienti privi di informazioni, come i labirinti, non è possibile applicare metodi esatti.

Come affrontare la risoluzione dei problemi

Per scegliere l'approccio corretto, bisogna considerare:

- **La complessità del problema**
- **La dimensione dell'input**
- **La struttura delle istanze (alcune possono essere più difficili di altre)**
- **Il tempo disponibile per trovare una soluzione**
- **I vincoli temporali (es. applicazioni real-time)**

Per problemi molto grandi, anche se polinomiali, si possono usare metaeuristiche. I vincoli temporali complicano ulteriormente il problema: serve trovare la **migliore soluzione possibile nel minor tempo possibile**.

Spazio di Ricerca

Il successo di un algoritmo dipende da come viene definito lo spazio di ricerca. Dato un problema combinatorio P , lo spazio di ricerca associato alla sua formulazione matematica è definito da una coppia (S, f) , dove:

- S è un insieme finito di configurazioni (detti anche nodi o punti);
- f è una funzione di costo che associa a ciascuna configurazione di S un numero reale.

In questa struttura, le due misure più comuni sono il costo minimo e il costo massimo. In tali casi, si parla di problemi di ottimizzazione combinatoria.

Chiavi di successo di un algoritmo

Per sviluppare un algoritmo efficace, è fondamentale prestare attenzione ai seguenti aspetti:

- Codifica del problema
- Rappresentazione della soluzione
- Definizione della funzione obiettivo
- Progettazione del metodo di ricerca della soluzione

Una volta formalizzato il problema, il passo cruciale è immaginare come rappresentare una soluzione. È importante ricordare che un algoritmo efficiente in domini continui potrebbe non funzionare altrettanto bene in domini discreti.

Il problema del Feedback Vertex Set (o Feedback Arc Set)

Si tratta di un problema computazionalmente complesso (NP-hard):

Dato un grafo con **n** vertici e **m** archi, si vuole rimuovere il minor numero di nodi tale da rendere il grafo aciclico.

Questa problematica trova applicazioni in ambito informatico, ad esempio nella sicurezza. Un aspetto rilevante è che l'ordine dei vertici influisce sulla soluzione trovata: permutazioni diverse possono generare soluzioni diverse.

Rappresentazione della soluzione

- **Intera**: l'algoritmo lavora su numeri interi
- **Binaria**: ogni vertice può essere rappresentato con 0 (rimosso) o 1 (presente)

Esempio

Supponiamo di voler massimizzare il numero di bit 1 in una sequenza binaria:

$$\max (\sum_i x_i)$$

Soluzioni possibili:

- **X₁** = 0101100
- **X₂** = 1001100
- **X₃** = 0111000

Tutte sono ottenute tramite scambi di bit e sono equivalenti in termini di funzione obiettivo.

La funzione obiettivo

La funzione obiettivo è il "faro" dell'algoritmo: guida la ricerca determinando quali soluzioni sono migliori.

Esempi:

- **TSP (Traveling Salesman Problem):** lunghezza totale del tour
- **Feedback Vertex Set:** numero di vertici rimossi
- **Problema dello zaino (Knapsack):** valore totale degli oggetti inseriti

Quando non è nota in partenza, la funzione obiettivo deve essere progettata con grande attenzione, poiché valuta la qualità delle soluzioni.

Funzione obiettivo e spazio di ricerca

Lo spazio di ricerca rappresenta tutte le soluzioni possibili, buone o cattive. La funzione obiettivo valuta la qualità di ciascuna soluzione, permettendo all'algoritmo di:

- **Scartare soluzioni scadenti**
- **Selezionare quelle promettenti**

Problemi di ottimizzazione con vincoli

In molti problemi, prima di valutare la bontà di una soluzione, bisogna verificare che essa rispetti i vincoli.

Esempio – TSP

- **Vincolo:** il grafo deve contenere tutti gli archi necessari per formare un ciclo.
- **Lo spazio di ricerca** è l'insieme di tutte le permutazioni dei nodi.
- Alcune di queste **permutazioni** saranno ammissibili, altre non ammissibili.

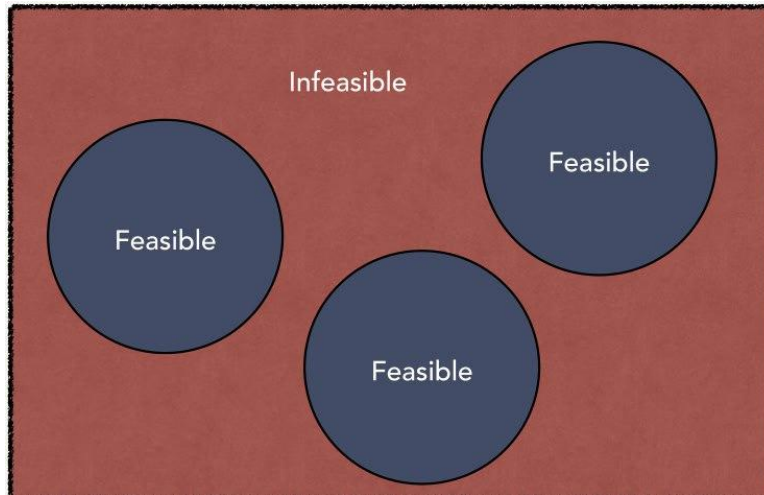
Due approcci possibili

1. Escludere a priori le soluzioni non valide

- ☒ Riduce lo spazio da esplorare
- ☒ Rischia di escludere la soluzione ottima (spesso si trova al confine della regione ammissibile)

2. Includere tutte le soluzioni (valide e non)

- ☒ Maggiore flessibilità ed esplorazione
- ☒ Necessita di un sistema per penalizzare le soluzioni non valide



Penalizzazione

Si può introdurre un fattore di penalità proporzionale alla gravità delle violazioni dei vincoli (es. numero di archi mancanti).

Una soluzione è valida quando non viola alcun vincolo.

Spazio di ricerca vs Spazio delle soluzioni

- **Spazio di ricerca:** contiene tutte le soluzioni possibili (valide e non valide)
- **Spazio delle soluzioni:** contiene solo le soluzioni valide

In assenza di vincoli, i due spazi coincidono.

In presenza di vincoli, lo spazio delle soluzioni è un sottoinsieme dello spazio di ricerca

Panorama combinatoriale (Search Landscape)

È una rappresentazione grafica dell'intero spazio di ricerca, dove:

- Ogni punto rappresenta una soluzione
- L'altezza del punto rappresenta il valore della funzione obiettivo

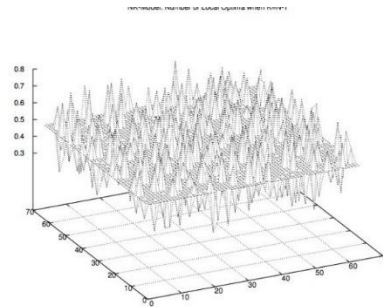
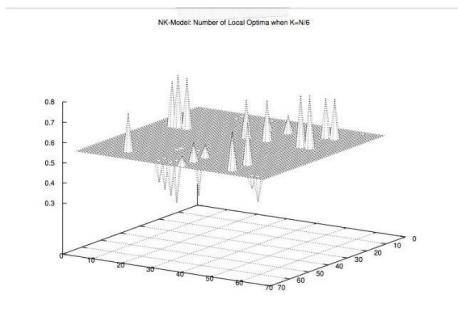
Serve a capire quanto è difficile un problema.

Componenti del panorama combinatoriale:

- Insieme S delle soluzioni ammissibili
- Funzione di fitness che assegna un valore reale a ciascuna soluzione
- Misura di distanza che definisce la distanza tra due soluzioni qualsiasi in S

Esempi di misure di distanza:

- Distanza di Hamming per stringhe binarie (numero di bit diversi)
- Distanza Euclidea per vettori continui



Analogia con il golf

- **Campo pianeggiante** = problema semplice (paesaggio "liscio")
- **Campo con dune, creste, valli** = problema difficile (paesaggio irregolare)

Proprio come un golfista adatta la strategia in base al campo, un algoritmo deve adattarsi alla forma del panorama del problema.

Tipologie di landscape

- **Facili:** poche colline, superfici lisce, nessun altopiano
- **Difficili:** molte colline, superfici discontinue, presenza di altopiani e creste

Teoria della complessità

Per determinare se un problema è davvero complesso, si fa riferimento alla teoria della complessità computazionale. Questa disciplina consente di capire se un problema può essere risolto in tempo polinomiale (quindi trattabile) oppure no (intrattabile).

Cos'è un problema

Un problema è una domanda generale a cui bisogna rispondere, solitamente con diversi parametri, o variabili libere, i cui valori sono lasciati non specificati.

Un problema è descritto fornendo:

- una descrizione generale di tutti i suoi parametri
- una dichiarazione delle proprietà che la risposta, o soluzione, deve soddisfare.

Un'istanza di un problema si ottiene specificando valori particolari per tutti i parametri del problema.

Esempio: il problema del commesso viaggiatore (TSP)

Un esempio classico: il problema del commesso viaggiatore (TSP)

I parametri di questo problema consistono in un insieme finito:

$C = c_1, c_2, \dots, c_m$ di "città" e, per ogni coppia di città c_i, c_j in C , la "distanza" $d(c_i, c_j)$ tra di esse.

Una soluzione è un ordinamento $c_{\pi(1)}, c_{\pi(2)}, \dots, c_{\pi(m)}$ delle città date che minimizza:

$$\sum_{i=1}^{m-1} d(c_{\pi(i)}, c_{\pi(i+1)}) + d(c_{\pi(m)}, c_{\pi(1)})$$

Questa espressione rappresenta la lunghezza del "tour" che parte da $c_{\pi(1)}$, visita ogni città in sequenza e poi ritorna direttamente a $c_{\pi(1)}$ partendo dall'ultima città $c_{\pi(m)}$.

Landscape di un problema

Il concetto di landscape è uno dei rari concetti esistenti che aiutano a comprendere il comportamento degli algoritmi di ricerca e delle euristiche e a caratterizzare la difficoltà di un problema combinatorio.

Esempio: problema K-SAT

Un problema K-SAT può essere rappresentato come un ipercubo di dimensione n , dove n è il numero di variabili booleane.

Esempio: $(A1 \wedge A2) \vee (A3 \wedge \neg A4)$

Clausole: $(A1 \wedge A2)$ e $(A3 \wedge \neg A4)$.

L'obiettivo è massimizzare il numero di clausole soddisfatte.

A1	A2	A3	A4	Clausole Soddisfatte
T	T	T	T	1
T	F	T	F	1
T	T	T	F	2

Algoritmi

Gli algoritmi sono procedure generali, passo dopo passo, per risolvere problemi.

Per essere concreti, possiamo pensarli semplicemente come programmi informatici, scritti in un linguaggio di programmazione preciso.

Si dice che un algoritmo risolva un problema P se può essere applicato a qualsiasi istanza I di P ed è sempre garantito che produca una soluzione per quell'istanza I .

Per esempio:

Un algoritmo non "risolve" il problema del TSP (commesso viaggiatore) a meno che non costruisca sempre un ordinamento che fornisca un tour di lunghezza minima.

Perché conoscere la teoria della NP-completezza

Se riusciamo a stabilire che un problema è NP-Completo, possiamo fornire una buona evidenza della sua intrattabilità.

Sarebbe meglio impiegare il nostro tempo nello sviluppo di un algoritmo di approssimazione piuttosto che nella ricerca di un algoritmo veloce che risolva il problema esattamente.

Problema: Punto iniziale

Un problema (astratto) è una relazione binaria su un insieme I di istanze del problema, e un insieme S di soluzioni del problema:

$$Q: I \rightarrow S$$

Esempio: Shortest-Path $\Rightarrow$ un'istanza è una tripla $(G, v_i - \text{sorgente}, v_j - \text{destinazione})$, mentre la soluzione è una sequenza di vertici nel grafo (percorso)

Una data istanza del problema può avere più di una soluzione.

Una **istanza** di un problema è una lista di argomenti, uno per ciascun parametro del problema

La teoria della NP-completezza limita l'attenzione ai **PROBLEMI DECISIONALI**: quelli che hanno una soluzione sì/no.

- Problema: si riferisce a una domanda come: "Un dato grafo $G=(V,E)$ è K -colorabile?"
- Problemi decisionali: $Q: I \rightarrow \{0, 1\}$

In generale, se possiamo risolvere rapidamente un problema di ottimizzazione, possiamo anche risolvere rapidamente il suo problema decisionale correlato. Se un problema di ottimizzazione è facile, allora anche il suo problema decisionale correlato è facile.

Definiamo un problema Q come una relazione binaria su un insieme I di istanze del problema e un insieme S di soluzioni del problema:

$$Q: I \rightarrow S$$

Problema del cammino minimo: trovare un percorso più breve tra due vertici dati in un grafo non pesato e non orientato $G=(V, E)$.

- **Istanza:** è una tripla composta da un grafo e due vertici.
- **Soluzione:** una sequenza di vertici nel grafo.

I cammini minimi non sono necessariamente unici: una data istanza del problema può avere più di una soluzione. Un'**ISTANZA** di un problema è una lista di argomenti, uno per ciascun parametro del problema.

La teoria della NP-completezza limita l'attenzione ai **problemi decisionali**: quelli con una soluzione sì/no.

Esempio: se $i = \langle G, k \rangle$ è un'istanza del problema di colorazione dei grafi, allora $COL(i) = 1$ (sì) se esiste una k -colorazione, e $COL(i) = 0$ (no) altrimenti.

Problemi di ottimizzazione: quelli in cui un valore deve essere minimizzato o massimizzato.

In generale, se possiamo risolvere velocemente un problema di ottimizzazione, possiamo risolvere velocemente anche il suo problema decisionale correlato

Problemi decidibili, indecidibili, trattabili e intrattabili

- **Decidibile**: esiste un algoritmo che risponde sempre sì/no correttamente.
- **Indecidibile**: nessun algoritmo risponde correttamente per ogni istanza.
- **Trattabile**: risolvibile in tempo polinomiale.
- **Intrattabile**: richiede tempo super polinomiale.

I problemi NP-completi sono intrattabili, se anche un solo problema NP-completo può essere risolto in tempo polinomiale, allora ogni problema NP-completo ammette un algoritmo a tempo polinomiale.

Se si dimostra che un problema è NP-completo, si fornisce una valida evidenza della sua intrattabilità. In tal caso, sarebbe meglio dedicare il proprio tempo a sviluppare un algoritmo approssimato piuttosto che cercare un algoritmo veloce che risolva il problema esattamente.

I problemi risolvibili in tempo polinomiale sono generalmente considerati trattabili.

Le classi P e NP

- **Classe P:** problemi risolvibili in tempo polinomiale.
- **Classe NP:** problemi la cui soluzione può essere verificata in tempo polinomiale.

Classe di complessità P

Un algoritmo risolve un problema in tempo $O(T(n))$ se, quando gli viene fornita un'istanza del problema i di lunghezza $n = |i|$, l'algoritmo può produrre la soluzione in al massimo $O(T(n))$

Un problema è risolvibile in tempo polinomiale se esiste un algoritmo che lo risolve in tempo $O(n^k)$ per qualche costante k

CLASSE DI COMPLESSITÀ P: l'insieme dei problemi decisionali risolvibili in tempo polinomiale

Una funzione f è calcolabile in tempo polinomiale se esiste un algoritmo A a tempo polinomiale che, dato un qualsiasi input x appartenente ad I , produce come output $f(x)$.

Non tutti i problemi sono risolvibili in tempo polinomiale, un esempio è l'**halting problem**, chiede se sia sempre possibile, descritto un algoritmo e un determinato ingresso finito, stabilire se l'algoritmo in questione termina o continua la sua esecuzione all'infinito.

Classe NP

Un problema decisionale (domanda sì/no) appartiene alla **classe NP** se possiede un algoritmo non deterministico a tempo polinomiale

Informalmente, un tale algoritmo:

1. Indovina una soluzione (in modo non deterministico).
2. Verifica deterministicamente in tempo polinomiale che la risposta sia corretta.

O equivalentemente, quando la risposta è "sì", esiste un certificato (una soluzione che soddisfa i criteri) che può essere verificato in tempo polinomiale (in modo deterministico). La **Classe di Complessità NP** è l'insieme dei problemi che possono essere verificati da un algoritmo a tempo polinomiale

$$\text{Se } Q \in P \text{ allora } Q \in NP \Rightarrow P \subseteq NP$$

La questione aperta è: **P = NP? No!**

Il problema del ciclo hamiltoniano

Un ciclo hamiltoniano in un grafo non orientato $G=(V,E)$ è un ciclo semplice che contiene ogni vertice di V .

Un grafo che contiene un ciclo hamiltoniano si dice hamiltoniano (NON TUTTI I GRAFI SONO HAMILTONIANI)

Problema del ciclo hamiltoniano: "Un grafo G possiede un ciclo hamiltoniano?"

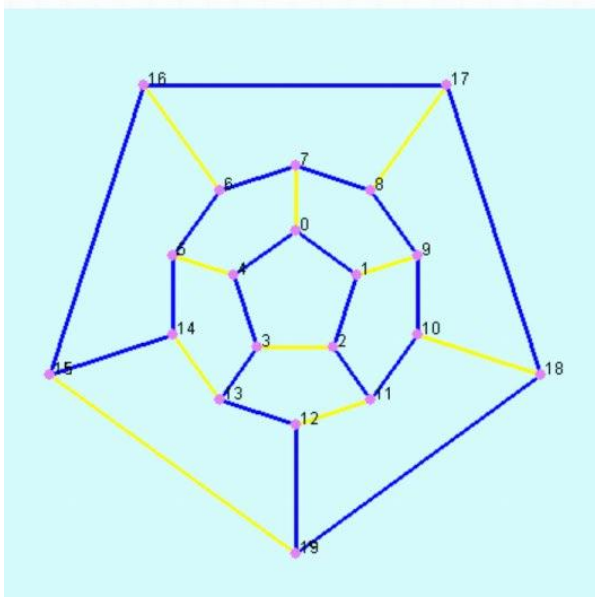
HAM-CYCLE = $\{ \langle G \rangle : G \text{ è un grafo hamiltoniano} \}$

Algoritmo: elenca tutte le permutazioni dei vertici e verifica per ciascuna permutazione se costituisce un percorso hamiltoniano: esistono **$m!$** possibili permutazioni dei vertici.

Verifica se il ciclo fornito è hamiltoniano: controlla se è una permutazione dei vertici di V e se ogni coppia consecutiva di vertici nel ciclo è collegata da un arco nel grafo. Questa verifica può essere implementata per essere eseguita in tempo **$O(n^2)$** => l'esistenza di un ciclo hamiltoniano in un grafo può essere verificata in tempo polinomiale. N è la lunghezza della codifica di G .

Il problema del ciclo hamiltoniano è un problema di classe NP

- **Hamiltonian Cycle** (input: a graph G)
- *Does G have a Hamiltonian cycle?*



Solution:

**0, 1, 2, 11, 10, 9, 8, 7, 6, 5, 14,
15, 6, 17, 18, 19, 12, 13, 3, 4**

Problemi NP-completi

La classe dei problemi in NP che sono i "più difficili" è chiamata **problemi NP-completi**

Un problema Q in NP è NP-completo se l'esistenza di un algoritmo a tempo polinomiale per Q implica l'esistenza di un algoritmo a tempo polinomiale per tutti i problemi in NP

Steve Cook nel 1971 dimostrò che SAT è NP-completo (Teorema di Cook-Levin)

Esempi:

- Colorazione di grafi
- Maximum independent set
- Hamiltonian cycle
- Clique

Riducibilità

Il concetto chiave della NP-Completezza è la **riducibilità**. Informalmente, un problema P può essere ridotto a un altro problema Q se ogni istanza di P può essere "facilmente riformulata" come un'istanza di Q, la cui soluzione fornisce una soluzione all'istanza di P. Questa riformulazione è chiamata **trasformazione**.

Intuitivamente: se P si riduce a Q, allora P è "non più difficile da risolvere" di Q

Se P è riducibile in tempo polinomiale a Q, lo denotiamo con $P \leq_p Q$

Definizione di NP-Completo:

- Se P è NP-Completo, allora $P \in \text{NP}$ e tutti i problemi R sono riducibili a P
- Formalmente: $R \leq_p P \forall R \in \text{NP}$

Se $P \leq_p Q$ e P è NP-Completo, allora anche Q è NP-Completo

NP-HARD ed NP-COMplete

Se P è riducibile in tempo polinomiale a Q , lo denotiamo con $P \leq_p Q$

Definizione di NP-Hard e NP-Completo:

- Se tutti i problemi $R \in NP$ sono riducibili a P , allora P è NP-Hard
- Diciamo che P è NP-Completo se P è NP-Hard e $P \in NP$

Se $P \leq_p Q$ e P è NP-Completo, allora anche Q è NP-Completo

Teorema

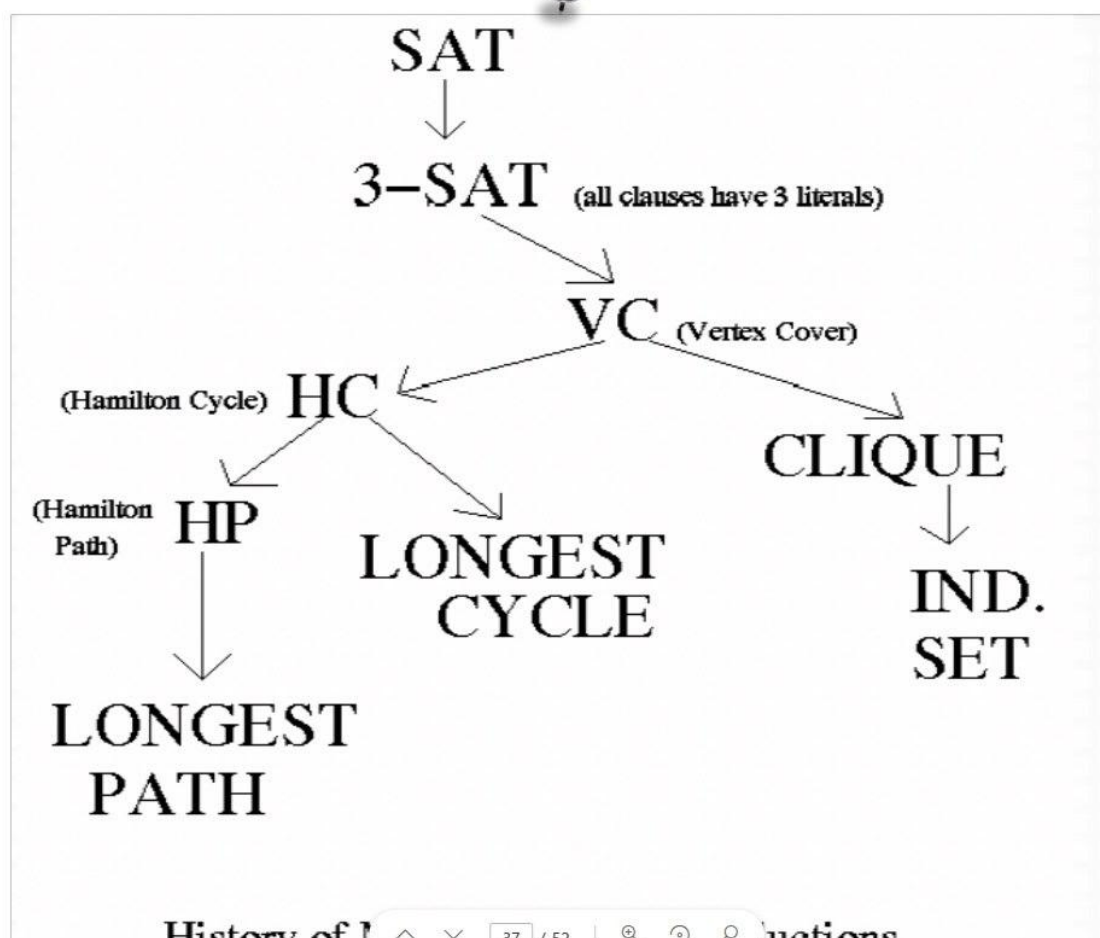
Definizione

Un problema B è detto NP-completo se:

1. $B \in NP$
2. Per ogni $A \in NP$ vale che $A \leq_p B$

Teorema

Se B è un problema NP-completo tale che $B \leq_p C$ per qualche problema C , allora B è NP-hard. Se poi $B \in NP$, allora vale che B è NP-completo.



SAT (Satisfiability)

Variabili: $(u_1, u_2, u_3, \dots, u_k)$

Un **letterale** è una variabile u_i o la sua negazione $\neg u_i$

Se u è impostata a vero, allora $\neg u$ è falso, e se u è impostata a falso, allora $\neg u$ è vero. Una **clausola** è un insieme di letterali. Una clausola è **vera** se almeno uno dei suoi letterali è **vero**

L'input per SAT è una collezione di clausole.

L'output è la risposta alla domanda: **"Esiste un'assegnazione di valori vero/falso alle variabili tale che ogni clausola risulti soddisfatta"** (una clausola è soddisfatta quando risulta vera)?"

Se la risposta è affermativa, tale assegnazione di valori alle variabili è chiamata assegnazione di verità.

SAT appartiene a NP: il certificato è l'assegnazione di valori vero/falso a ciascuna variabile che soddisfa tutte le clausole.

ESEMPIO:

$$(x_1 \vee x_2 \vee \overline{x}_3) \wedge (\overline{x}_1 \vee \overline{x}_2 \vee \overline{x}_3) \wedge (x_1 \vee \overline{x}_2) \wedge (\overline{x}_1 \vee \overline{x}_2)$$

- La formula é soddisfacibile, semplicemente ponendo $x_1=1$, $x_2=0$, e $x_3=0$.
- Infatti

$$(1 \vee 0 \vee 1) \wedge (0 \vee 1 \vee 1) \wedge (1 \vee 1) \wedge (0 \vee 1) = 1$$

- La seguente formula, invece, non é soddisfacibile

$$(x_1 \vee x_2) \wedge (x_1 \vee x_3) \wedge (x_2 \vee x_3) \wedge (\overline{x}_1 \vee \overline{x}_2) \wedge (\overline{x}_1 \vee \overline{x}_3) \wedge (\overline{x}_2 \vee \overline{x}_3)$$

Teorema di Cook-Levin

SAT è NP-Completo

Dimostrazione

In sintesi, il teorema di Cook-Levin dimostra che:

- Il problema SAT appartiene a NP e può essere deciso da un opportuno modello computazionale: di (NTM)
- Data la descrizione di una NTM n e un input w per n , è possibile costruire un'espressione in forma normale congiuntiva Φ_w soddisfacibile se e solo se l'output di n sull'input w è accettante
- La lunghezza della formula Φ_w è polinomiale in $|n|$ e $|w|$

Riduzione

Se SAT è risolvibile in tempo polinomiale, allora lo è anche HAMILTON CYCLE

Il grafo G ha n vertici: $0, 1, 2, \dots, n-1$

- **Variabili:** $x_{i,j}$ ($0 \leq i, j \leq n-1$)
- **Significato:** il nodo i occupa la posizione j nel ciclo hamiltoniano

Condizioni per garantire un ciclo hamiltoniano

1. Esattamente un nodo compare nella posizione j

Almeno un nodo deve apparire nella posizione j

Per ogni j , aggiungere una clausola:

$$(x_{0,j} \text{ OR } x_{1,j} \text{ OR } x_{2,j} \text{ OR } \dots \text{ OR } x_{n-1,j})$$

Al più un nodo può comparire nella posizione j

Per ogni coppia di vertici i, k aggiungere una clausola

$$(\text{not } x_{i,j} \text{ OR } \text{not } x_{k,j})$$

2. Il vertice i compare esattamente una volta nel ciclo

Il vertice i compare almeno una volta.

Per ogni i, aggiungere una clausola:

$$(x_{i,0} \text{ OR } x_{i,1} \text{ OR } x_{i,2} \text{ OR } \dots \text{ OR } x_{i,n-1})$$

Il vertice i compare al più una volta

Per ogni vertice i e coppia di posizioni j, k aggiungere una clausola

$$(\text{not } x_{i,j} \text{ OR not } x_{i,k})$$

3. I vertici consecutivi del ciclo sono connessi da un arco del grafo

Per ogni arco (i,k) che è assente dal grafo e per ogni j, aggiungi una clausola:

$$(\text{not } x_{i,j} \text{ OR not } x_{k,j+1 \bmod n})$$

Traveling Salesman Problem

Il noto problema del commesso viaggiatore:

- **Variante di ottimizzazione:** un commesso deve visitare n città, visitando ciascuna città esattamente una volta e tornando al punto di partenza.
Come minimizzare il tempo di viaggio?
- Modella il problema come un grafo completo con costo $c(i,j)$ per andare dalla città i alla città j.
- Come possiamo trasformarlo in un problema decisionale?
 - chiedere se **esiste** un TSP con costo $< k$

I passaggi per dimostrare che il TSP è **NP-Completo**:

- Dimostrare che **TSP $\in$ NP**
- Ridurre il problema del **ciclo hamiltoniano non orientato** al TSP
 - Quindi, se avessimo un risolutore per il TSP, potremmo usarlo per risolvere il problema del ciclo hamiltoniano in **tempo polinomiale**
 - Come possiamo trasformare un'istanza del problema del ciclo hamiltoniano in un'istanza del TSP?
 - Possiamo farlo in tempo polinomiale?

Clique

ESEMPIO: CLIQUE

ISTANZA: Un grafo $G = (V, E)$ ed un intero k

DOMANDA: G ha una clique di taglia k ?

(ovvero $\exists V' \subseteq V, |V'| = k$, tale che $\forall u, v \in V', u \neq v$ vale che $(u, v) \in E$)

Teorema

CLIQUE è NP-completo

Dimostrazione

Proveremo che **CLIQUE $\in$ NP**, successivamente proveremo che

3SAT $\leq_p$ CLIQUE. Dal Teorema 1 discenderà che **CLIQUE è NP-completo**.

Un algoritmo di verifica per il problema **CLIQUE** è il seguente: avendo in input la coppia (G, k) e come certificato un sottoinsieme $X \subseteq V$, con $|X| = k$ (potenziale soluzione al problema **CLIQUE**), l'algoritmo controlla tutte le coppie di vertici (u, v) , con $u, v \in X, u \neq v$, e produce in output **SI** se e solo se ciascuna di queste coppie è connessa da un arco di E . Il tempo di esecuzione dell'algoritmo è $O(n^2)$, se $n = |V|$.

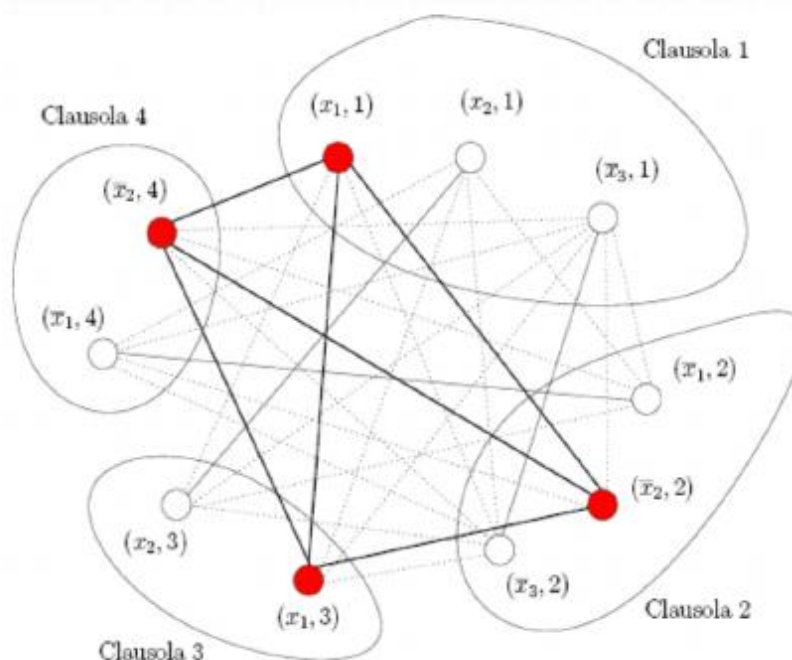
3SAT riducibile in tempo polinomiale a CLIQUE

3SAT $\leq_p$ CLIQUE

- Sia $\phi = C_1 \wedge C_2 \wedge \dots \wedge C_k$ una istanza di 3SAT in una istanza (G_ϕ, k) di Clique come segue:
 1. Per ogni letterale scriviamo un nodo
 2. Colleghiamo tutti i nodi ad eccezione:
 - Nodi nella medesima clausola
 - Nodi che rappresentano letterali opposti

ESEMPIO

$$(x_1 \vee x_2 \vee \bar{x}_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3) \wedge (x_1 \vee x_2) \wedge (\bar{x}_1 \vee \bar{x}_2)$$



Vertex Cover

ESEMPIO: VERTEX-COVER

ISTANZA: Un grafo $G = (V, E)$ ed un intero k

DOMANDA: esiste $V' \subseteq V$, $|V'| = k$, tale che $\forall (u, v) \in E$ o vale che $u \in V'$ oppure $v \in V'$?

Teorema

Vertex-Cover é NP-completo

Dimostrazione

Proveremo che Vertex-Cover $\in$ NP, successivamente proveremo che $3SAT \leq_p$ Vertex-Cover. Dal Teorema 1 discenderà che Vertex-Cover é NP-completo.

Che il problema VERTEX COVER sia in NP é ovvio. Un algoritmo di verifica prende in input l'istanza di input (G, k) ed un certificato $V' \subseteq V$, $|V'| = k$, e controlla se

$\forall (u, v) \in E$ vale che $u \in V'$ oppure che $v \in V'$. Il controllo può essere fatto in tempo $O(n^2)$.

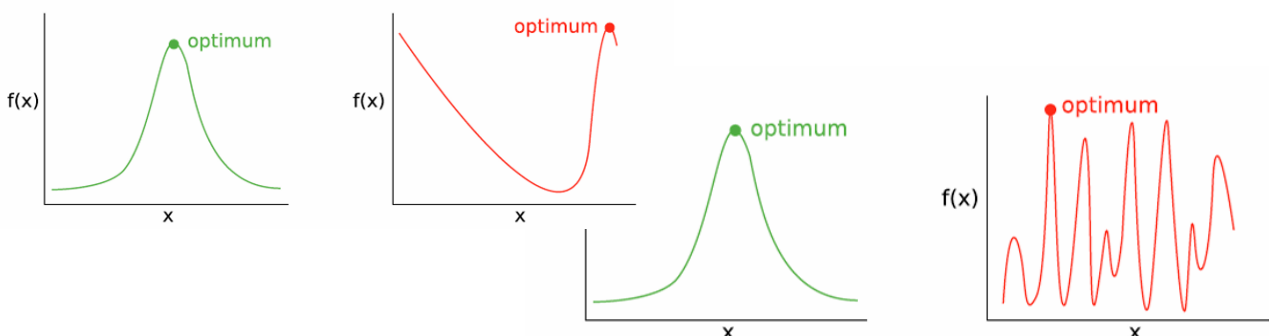
Ottimi Locali vs Ottimi Globali

Funzioni semplici

- **"Essere più vicini a un ottimo globale porta a una maggiore fitness"**
 - Più ci si avvicina alla soluzione ideale, più la qualità (fitness) migliora. C'è una correlazione diretta e stabile tra vicinanza all'ottimo e valore della fitness.
- **"Salire direttamente verso l'ottimo è piuttosto facile"**
 - Ci sono pochi ostacoli o trappole lungo il cammino verso l'ottimo: il gradiente (direzione di miglioramento) guida correttamente verso la soluzione.
- **"Soluzioni vicine tra loro hanno valori di fitness simili"**
 - Il paesaggio è liscio o regolare: piccole modifiche a una soluzione non cambiano drasticamente il risultato.
- **"Le differenze di fitness aumentano lentamente con la distanza"**
 - Allontanandosi dall'ottimo, la fitness peggiora gradualmente, senza bruschi salti.

Funzioni difficili

- **"Essere più vicini a un ottimo globale può portare a una fitness inferiore"**
 - La funzione è ingannevole: puoi trovarti vicino all'ottimo vero, ma con un valore di fitness peggiore rispetto a zone più lontane. Quindi la fitness non è una guida affidabile.
- **"Salire direttamente verso l'ottimo non è così facile"**
 - Il paesaggio ha molti massimi locali o "colline sbagliate", quindi seguire il gradiente può portare fuori strada.
- **"Soluzioni vicine tra loro hanno valori di fitness differenti"**
 - Il paesaggio è irregolare o frastagliato: piccole modifiche possono avere grandi effetti, rendendo l'ottimizzazione instabile.
- **"Le differenze di fitness aumentano rapidamente con la distanza"**
 - Spostamenti nello spazio delle soluzioni possono comportare variazioni drastiche nella fitness.



Cos'è l'ottimizzazione vincolata?

Il problema generale che abbiamo considerato qui può essere descritto come:

$$\min_x \{f(x)\}$$

soggetto a

$$g_i(x) \leq 0, \quad i = 1, 2, \dots, m$$

$$h_j(x) = 0, \quad j = 1, 2, \dots, p$$

Dove:

- x è un vettore di dimensione n , $x = (x_1, x_2, \dots, x_n)$;
- $f(x)$ è la funzione obiettivo;
- $g_i(x)$ è il vincolo di disuguaglianza;
- $h_j(x)$ è il vincolo di uguaglianza.

Indichiamo l'intero spazio di ricerca con S e lo spazio ammissibile con $\mathcal{F}$, con $\mathcal{F} \subset S$.

È importante notare che il minimo globale in $\mathcal{F}$ potrebbe non essere lo stesso che in S .

Tecniche di Gestione dei Vincoli

- L'approccio della **funzione di penalità** converte un problema vincolato in uno non vincolato introducendo una funzione di penalità nella funzione obiettivo.
- L'approccio di **riparazione** trasforma (ripara) una soluzione non ammissibile in una ammissibile.
- L'approccio **purista** rifiuta tutte le soluzioni non ammissibili durante la ricerca.
- L'approccio **separatista** considera separatamente la funzione obiettivo e i vincoli.
- L'approccio **ibrido** combina due o più tecniche di gestione dei vincoli diverse.

Approccio delle Funzioni di Penalità

$$\text{Nuova Funzione Obiettivo} = \text{Funzione Obiettivo Originale} + \text{Coefficiente Di Penalità} * \text{Grado Di Violazione Dei Vincoli}$$

La forma generale del metodo della funzione di penalità esterna è:

$$\psi(x) = f(x) + \left(\sum_{i=1}^m r_i G_i(x) + \sum_{j=1}^p c_j H_j(x) \right)$$

Dove:

- $\psi(x)$ è la nuova funzione obiettivo da minimizzare,
- $f(x)$ è la funzione obiettivo originale,
- r_i e c_j sono fattori di penalità (coefficienti),

E:

$$G_i(x) = (\max(0, g_i(x)))^\beta, \quad H_j(x) = \max(0, |h_j(x)|^\gamma)$$

I parametri β e γ sono solitamente scelti uguali a 1 o 2.

- **Penalità Statiche:** La funzione di penalità è predefinita e fissa durante l'evoluzione.
- **Penalità Dinamiche:** La funzione di penalità cambia secondo una sequenza predefinita, che spesso dipende dal numero di generazione.
- **Penalità Adattive e Auto-Adattive:** La funzione di penalità cambia in modo adattivo. Non esiste una sequenza fissa da seguire.

Funzioni di Penalità Statiche

$$\psi(x) = f(x) + \sum_{i=1}^m r_i (G_i(x))^2$$

dove r_i sono predefiniti e fissi.

I vincoli di uguaglianza possono essere convertiti in vincoli di disuguaglianza:

$$h_j(\mathbf{x}) \Rightarrow |h_j(\mathbf{x})| - \varepsilon \leq 0$$

dove $\varepsilon > 0$ è un numero piccolo.

- Semplice e facile da implementare.
- Richiede una buona conoscenza del dominio per impostare i valori di r_i .
- I valori di r_i possono essere suddivisi in diversi livelli. Quando utilizzare ciascuno di essi è determinato da un insieme di regole euristiche.

Funzioni di Penalità Dinamiche

Principio generale: Maggiore è il numero della generazione, maggiore è il coefficiente di penalità. Perché?

Metodo di Joines e Houck:

$$\psi(x) = f(x) + (Ct)^\alpha \left(\sum_{i=1}^m |g_i(x)|^\beta + \sum_{j=1}^p |h_j(x)|^\beta \right)$$

dove C , α e β sono costanti definite dall'utente, ad esempio:

$C = 0.5$, $\alpha = 1, 2$, $\beta = 2$, e t è il numero della generazione.

- **Possibili variazioni:** Esistono molte forme diverse di penalità dinamiche, ad esempio $\frac{t}{T}$. Trovare la migliore può essere difficile.
- **Coefficienti differenziati:** È possibile utilizzare coefficienti diversi per vincoli di disuguaglianza e uguaglianza.

La **funzione di penalità** è definita come:

$$\psi(x) = f(x) + r(t) \sum_{i=1}^m G_i^2(x) + c(t) \sum_{j=1}^p h_j^2(x),$$

dove $r(t)$ e $c(t)$ sono due coefficienti di penalità dipendenti dal tempo (o dall'iterazione t).

Possibili Forme per $r(t)$ e $c(t)$:

1. Forma Polinomiale

I parametri a_i e b_i sono definiti dall'utente:

$$r(t) = a_0 + a_1 t + a_2 t^2 + \dots, \quad c(t) = b_0 + b_1 t + b_2 t^2 + \dots$$

2. Forma Esponenziale

I parametri a e b sono definiti dall'utente:

$$r(t) = e^{at}, \quad c(t) = e^{bt}$$

3. Forma Ibrida (Polinomiale + Esponenziale)

$$r(t) = e^{a_0 + a_1 t + a_2 t^2 + \dots}, \quad c(t) = e^{b_0 + b_1 t + b_2 t^2 + \dots}$$

Osservazioni

- **Adattamento nel tempo:** I coefficienti crescono con t , aumentando la penalizzazione man mano che l'ottimizzazione procede.
- **Flessibilità:** La scelta tra polinomi, esponenziali o forme ibride dipende dal problema e dall'esperienza dell'utente.
- **Controllo della convergenza:** Una crescita troppo rapida (es. esponenziale) può portare a soluzioni troppo conservative, mentre una crescita lenta (es. lineare) può non penalizzare abbastanza.

Funzioni di Penalità Adattive

Metodo di Bean e Hadj-Alouane

$$\psi(x) = f(x) + \lambda(t) \left(\sum_{i=1}^m G_i^2(x) + \sum_{j=1}^p |h_j(x)| \right)$$

dove λ viene aggiornato alla generazione t come segue:

$$\lambda(t) = \begin{cases} \frac{1}{\beta_1} \lambda(t), & \text{if case 1,} \\ \beta_2 \lambda(t), & \text{if case 2,} \\ \lambda(t), & \text{otherwise,} \end{cases}$$

dove caso 1 (o 2) indica che il miglior individuo nelle ultime k generazioni era sempre fattibile (o infattibile), con $\beta_2 > \beta_1 > 1$.

Funzione di Fitness e Selezione

Sia $\Phi(x) = f(x) + rG(x)$ dove $G(x) = \sum_{i=1}^m G_i(x)$ e $G_i(x) = \max\{0, g_i(x)\}$

Dati due individui x_1 e x_2 , i loro valori di fitness saranno determinati da $\Phi(x)$.

Poiché i valori di fitness sono utilizzati principalmente nella selezione

Modificare il fitness $\Leftrightarrow$ modificare le probabilità di selezione

When r Plays an Important Role

$\Phi(x_1) < \Phi(x_2)$

means

$$f(x_1) + rG(x_1) < f(x_2) + rG(x_2).$$

1. $f(x_1) < f(x_2)$ and $G(x_1) < G(x_2)$: r has no impact on the comparison.
2. $f(x_1) < f(x_2)$ and $G(x_1) > G(x_2)$: Increasing r will eventually change the comparison.
3. $f(x_1) > f(x_2)$ and $G(x_1) < G(x_2)$: Decreasing r will eventually change the comparison.

In essence, different r 's lead to different rankings of individuals in the population.

Algoritmo di Riparazione

- **Seleziona un individuo di riferimento** I_r (tipicamente un individuo ammissibile).
- **Genera una sequenza di individui candidati** z_i tra I_s (individuo infeasibile da riparare) e I_r :

$$z_i = a_i I_s + (1 - a_i) I_r,$$

dove $0 < a_i < 1$ può essere generato casualmente o deterministicamente.

Il processo si interrompe quando z_i diventa ammissibile (cioè, quando viene trovato il primo z_i feasible).

- **Assegna** questo z_i come versione riparata di I_s . Il valore della funzione obiettivo di z_i verrà utilizzato come fitness di I_s .
- **Sostituisci** I_s con z_i con probabilità P_r (anche se I_s non viene sostituito, il suo fitness rimane quello di z_i)
- **Aggiornamento dell'individuo di riferimento:** Se z_i è migliore di I_r , sostituisci I_r con z_i

Algoritmo di Riparazione: Problemi implementativi

COME TROVARE GLI INDIVIDUI DI RIFERIMENTO INIZIALI?

- Esplorazione preliminare
- Conoscenza umana

COME SELEZIONARE I_r ?

- Casualmente in modo uniforme
- In base al fitness di I_r
- In base alla distanza tra I_r e I_s

COME DETERMINARE a_i ?

- Casualmente tra 0 e 1
- Usando una sequenza fissa, es. $\frac{1}{2}, \frac{1}{4}, \dots$

COME SCEGLIERE P_r ?

Un numero piccolo, solitamente < 0.5 .

Problemi complessi

Ottimo Locale

Uno degli ostacoli principali per un algoritmo di ottimizzazione è la presenza di ottimi locali: soluzioni che appaiono ottimali in un intorno limitato dello spazio di ricerca, ma che non rappresentano il miglior risultato possibile (cioè, l'ottimo globale). L'algoritmo, confuso dalla qualità apparente di queste soluzioni, rischia di fermarsi prematuramente, compromettendo il risultato finale.

Come esplorare lo spazio di ricerca (chiave del successo)

Un elemento cruciale per il successo di un algoritmo è la strategia di esplorazione dello spazio di ricerca, soprattutto in ambienti complessi o dinamici. Spesso si lavora in condizioni di incertezza, con informazioni incomplete o in continua evoluzione. L'algoritmo deve quindi essere in grado di muoversi "alla cieca", affrontando l'ambiente anche in assenza di guida esplicita.

Vincoli temporali e quantità di informazioni

Altri ostacoli importanti sono:

- **Vincoli temporali:** anche problemi appartenenti alla classe polinomiale possono diventare intrattabili se il tempo a disposizione è limitato.
- **Elevate quantità di dati:** aumentano il carico computazionale e richiedono algoritmi efficienti per essere gestiti in tempo utile.

Ispirazione dalla natura

Quando non si conosce un metodo risolutivo efficace, può essere utile osservare la natura, che offre sistemi estremamente robusti e adattivi. Ad esempio, gli alberi si adattano al contesto in cui crescono. Tuttavia, non basta copiare la natura: è essenziale comprendere cosa osservare e cosa estrapolare da essa.

Teoria dell'evoluzione

La teoria dell'evoluzione spiega come, nel tempo, gli organismi si siano evoluti verso forme più robuste e adatte all'ambiente. Esempio emblematico: le giraffe. Inizialmente avevano il collo corto e sopravvivevano con difficoltà; nel tempo, l'evoluzione ha favorito individui con colli più lunghi, capaci di accedere meglio al cibo, che hanno avuto maggior successo riproduttivo.

Concetti fondamentali:

- **Selezione naturale:** sopravvive chi si adatta meglio.
- **Adattamento:** evoluzione di tratti favorevoli.
- **Crossover:** combinazione del patrimonio genetico dei genitori.
- **Mutazione:** cambiamenti casuali che introducono variabilità.
- **Evoluzione:** trasmissione di tratti robusti alle generazioni successive.

Algoritmi Bio & Natural Inspired

- **Algoritmi genetici**
- **Algoritmi evolutivi**

Questi algoritmi si ispirano ai meccanismi naturali di evoluzione per trovare soluzioni a problemi complessi.

Swarm Intelligence (Intelligenza dello sciame)

Algoritmi ispirati al comportamento collettivo e decentralizzato osservato in natura (formiche, api, stormi di uccelli). Ogni individuo agisce in modo autonomo, senza una guida centrale, ma l'interazione collettiva genera comportamenti complessi e ordinati.

Esempio: anche quando guidiamo individualmente, il sistema complessivo può risultare ordinato e funzionale, pur senza coordinazione centralizzata.

Una semplice azione individuale, replicata e combinata in gruppo, può generare un sistema emergente complesso.

Colonie di formiche

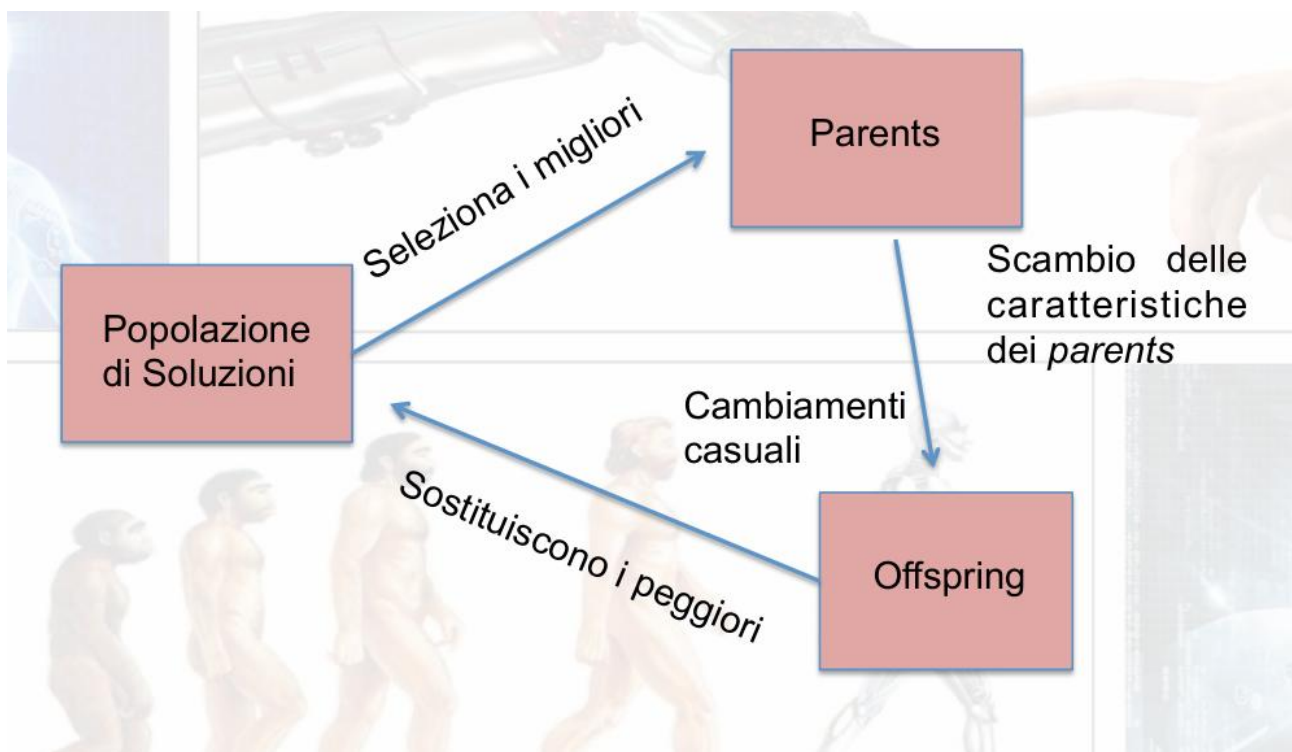
Le formiche, pur prive di una mappa dell'ambiente, riescono a trovare il cammino più breve verso il cibo grazie a meccanismi di esplorazione e comunicazione tramite feromoni. Dopo una prima fase di esplorazione casuale, emergono percorsi ottimali, capaci di adattarsi anche a variazioni dell'ambiente.

L'obiettivo è riprodurre artificialmente questo tipo di comportamento emergente.

Funzionamento dei sistemi evolutivi

Nel modello evolutivo:

- Ogni individuo rappresenta una possibile soluzione.
- L'ambiente rappresenta il problema da risolvere.
- Gli individui nascono, crescono e competono per la sopravvivenza.
- Alcuni diventano genitori, generano figli (offspring), che a loro volta entrano nella popolazione.
- Questo processo forma un ciclo evolutivo continuo.



Perché sviluppare una nuova classe di algoritmi?

Questi algoritmi:

- Lavorano in modo stocastico, non deterministico.
- Non costruiscono la soluzione elemento per elemento, ma partono da soluzioni casuali, migliorandole iterativamente.
- Non assumono che una buona scelta oggi sarà buona anche in futuro: migliorano per successivi raffinamenti.

Confronti

Deterministico versus stocastico:

Una metaeuristica deterministica risolve un problema di ottimizzazione prendendo decisioni deterministiche (ad esempio, ricerca locale, tabu search). Nelle metaeuristiche stocastiche, durante la ricerca vengono applicate alcune regole casuali (ad esempio, simulated annealing, algoritmi evolutivi). Negli algoritmi deterministici, utilizzando la stessa soluzione iniziale si otterrà la stessa soluzione finale, mentre nelle metaeuristiche stocastiche, partendo dalla stessa soluzione iniziale si possono ottenere soluzioni finali diverse. Questa caratteristica deve essere presa in considerazione nella valutazione delle prestazioni degli algoritmi metaeuristici.

Iterativo versus greedy

Negli algoritmi iterativi, si inizia con una soluzione completa (o una popolazione di soluzioni) e la si trasforma ad ogni iterazione utilizzando alcuni operatori di ricerca. Gli algoritmi greedy partono da una soluzione vuota e, ad ogni passo, viene assegnata una variabile decisionale del problema fino a ottenere una soluzione completa. La maggior parte delle metaeuristiche sono algoritmi iterativi.

Ricerca basata su popolazione versus ricerca basata su singola soluzione:

Gli algoritmi basati su singola soluzione (ad esempio, ricerca locale) manipolano e trasformano una singola soluzione durante la ricerca, mentre negli algoritmi basati su popolazione (ad esempio, particle swarm, algoritmi evolutivi) viene evoluta un'intera popolazione di soluzioni. Queste due famiglie hanno caratteristiche complementari: le metaeuristiche basate su singola soluzione sono orientate all'esplorazione; hanno il potere di intensificare la ricerca in regioni locali.

Vantaggi

- **Stocasticità:** l'intero processo si basa su scelte casuali.
- **Black-box:** non necessitano di conoscenze specifiche del dominio.
- **Adattabilità:** apprendono solo ciò che scoprono ("I know it when I see it").
- **Robustezza:** affrontano ambienti incerti con successo.
- **Decentralizzazione:** operano senza un controllo centrale.
- **Autonomia:** non richiedono interventi esterni durante l'esecuzione.
- **Flessibilità:** applicabili a diversi tipi di problemi.
- **Adattività:** capaci di modificare autonomamente il proprio comportamento in ambienti dinamici, senza la necessità di essere riavviati.

Metodi risolutivi dei problemi

I metodi per risolvere i problemi si dividono in due categorie principali:

- Esatti
- Approssimati

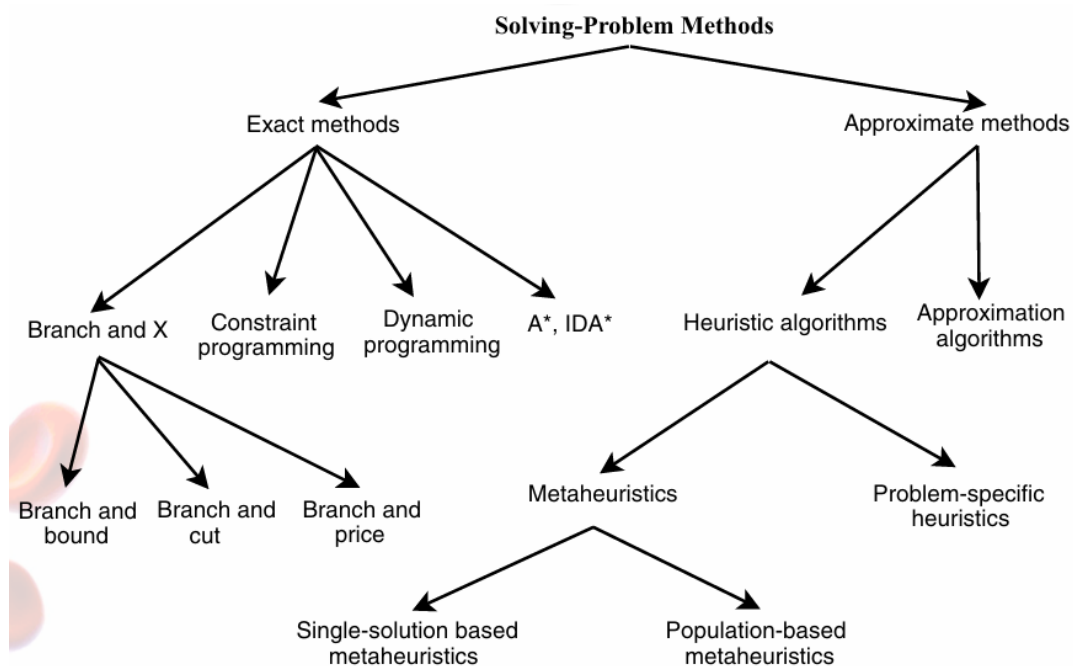


FIGURE: Classical Solving-Problem Methods

Punti Chiave

I punti chiave da considerare nella progettazione di un metodo risolutivo sono:

- La codifica del problema
- La rappresentazione della soluzione
- Il processo di ricerca della soluzione
- La definizione della funzione obiettivo

A seconda di come rappresentiamo la soluzione, cambia anche lo spazio di ricerca. La funzione obiettivo è essenziale per guidare l'algoritmo verso la soluzione desiderata.

Algoritmi Greedy

Gli algoritmi greedy (o avidi) costruiscono una soluzione passo dopo passo, partendo da una soluzione vuota e senza possibilità di backtracking (non si torna indietro).

Algorithm 1.2 Template of a greedy algorithm.

```
s = {} ; /* Initial solution (null) */  
Repeat  
     $e_i = \text{Local-Heuristic}(E \setminus \{e/e \in s\})$  ;  
    /* next element selected from the set  $E$  minus already selected elements */  
    If  $s \cup e_i \in F$  Then /* test the feasibility of the solution */  
         $s = s \cup e_i$  ;  
Until Complete solution found
```

Caratteristiche principali:

- Scelta locale ottima a ogni passo.
- Non garantiscono la ottimalità globale, solo quella locale.
- Sono detti algoritmi miopi, poiché considerano solo l'immediato intorno della soluzione corrente.
- Sono meno efficaci man mano che aumenta la complessità del problema.

Algoritmi Greedy - Limitazioni

Nella maggior parte dei problemi di ottimizzazione, le euristiche greedy, a causa della loro visione locale, tendono a ridurre le prestazioni rispetto agli algoritmi iterativi.

Ad ogni passo, viene utilizzata un'euristica per selezionare l'elemento successivo da includere nella soluzione. La scelta ricade sull'elemento che offre il miglior contributo locale all'ottimizzazione della funzione obiettivo.

Tuttavia, l'ottimalità locale non implica necessariamente l'ottimalità globale.

Le euristiche greedy sono generalmente utilizzate per la costruzione incrementale di una soluzione.

Algoritmi Iterativi

Gli algoritmi iterativi sono metodi approssimati che migliorano una soluzione iniziale tramite trasformazioni locali, spostandosi nel vicinato fino al soddisfacimento di un criterio di arresto.

Si applicano in modo iterativo le procedure di generazione delle soluzioni e sostituzione partendo dall'attuale singola soluzione tramite **trasformazioni locali**, percorrendo i vicinati finché non viene raggiunto un criterio di arresto prestabilito.

Algorithm 2.1 High-level template of S-metaheuristics.

Input: Initial solution s_0 .

$t = 0$;

Repeat

/* Generate candidate solutions (partial or complete neighborhood) from s_t */

Generate($C(s_t)$);

/* Select a solution from $C(s)$ to replace the current solution s_t */

$s_{t+1} = \text{Select}(C(s_t))$;

$t = t + 1$;

Until Stopping criteria satisfied

Output: Best solution found.

Due operazioni fondamentali:

1. **Generazione** di una soluzione iniziale
2. **Sostituzione** con una soluzione migliore trovata nel vicinato

I concetti fondamentali della ricerca sono la definizione della struttura del vicinato e la determinazione della soluzione iniziale. La struttura del vicinato gioca un ruolo cruciale nelle prestazioni di un algoritmo. Se la struttura del vicinato non è adeguata al problema, qualsiasi algoritmo fallirà nel risolverlo.

Concetti chiave:

- **Soluzione iniziale:** influenza fortemente le prestazioni.
- **Vicinato:** struttura e qualità del vicinato sono determinanti per l'efficacia dell'algoritmo.
- Alcuni algoritmi lavorano su una singola soluzione, altri su un insieme.

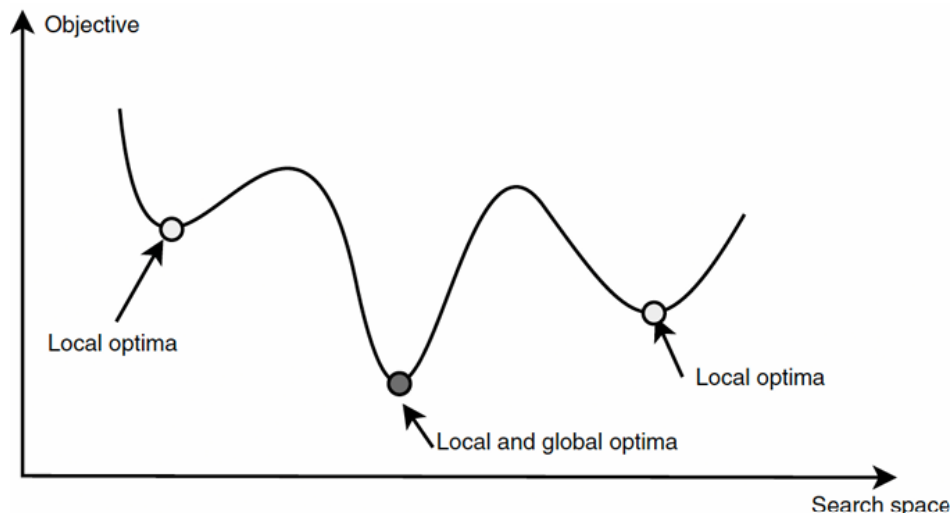
Strategie:

- Spesso è preferibile generare una soluzione iniziale casuale anziché deterministica.
- Esistono algoritmi con memoria (evitano di esplorare soluzioni già viste) e algoritmi senza memoria (possono riesplorare le stesse soluzioni).

Ottimo Locale – Definizione

Relativamente a una determinata funzione di vicinato N , una soluzione $s \in S$ è un ottimo locale se ha una qualità migliore di tutti i suoi vicini; cioè, $f(s) \leq f(s')$ per ogni $s' \in N(s)$

Importante: l'ottimo locale dipende dal vicinato considerato. Una soluzione può essere ottima in un vicinato N_1 ma non in un altro N_2 .



Soluzione Iniziale

La scelta della soluzione iniziale è cruciale. La qualità dell'ottimo locale ottenuto dal metodo di risoluzione considerato dipende dalla soluzione iniziale.

In molti algoritmi iterativi (come quelli di ricerca locale), si parte da una soluzione iniziale per esplorare il vicinato alla ricerca di soluzioni migliori.

Se la soluzione iniziale è "vicina" a un ottimo locale buono, l'algoritmo ha maggiori probabilità di trovare una soluzione di qualità. Al contrario, partire da una soluzione iniziale scadente può portare l'algoritmo a convergere verso ottimi locali poco soddisfacenti. Quindi, una buona scelta iniziale può migliorare significativamente le prestazioni dell'algoritmo.

Definizione di Vicinato

Una funzione di vicinato N è una mappatura $N: S \rightarrow 2^S$ che associa a ogni soluzione s di S un insieme di soluzioni $N(s) \subset S$.

Proprietà fondamentale: località

L'effetto sulla soluzione quando si esegue la perturbazione (mossa) nella rappresentazione. Quando vengono apportate piccole modifiche nella rappresentazione, la soluzione deve mostrare piccoli cambiamenti.

La definizione di vicinato dipende fortemente dalla rappresentazione scelta per il problema da risolvere.

Nel caso di rappresentazione binaria:

Il vicinato naturale è basato sulla distanza di Hamming. Per un vettore binario di dimensione n , la dimensione del vicinato è pari a n (ossia il numero di bit che possono essere "flippati" individualmente per generare nuove soluzioni).

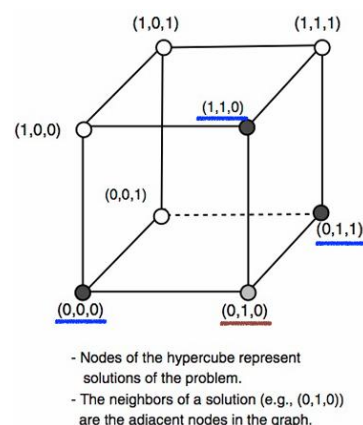
Nel caso di rappresentazione tramite permutazioni:

Per una permutazione di n elementi, la dimensione del vicinato è $\frac{n(n-1)}{2}$, se si utilizza il classico operatore di swap, che scambia la posizione di due elementi qualsiasi nella permutazione.

Esempio in rappresentazione binaria

Dati:

- $X = 10011$
- $Y = 11011$

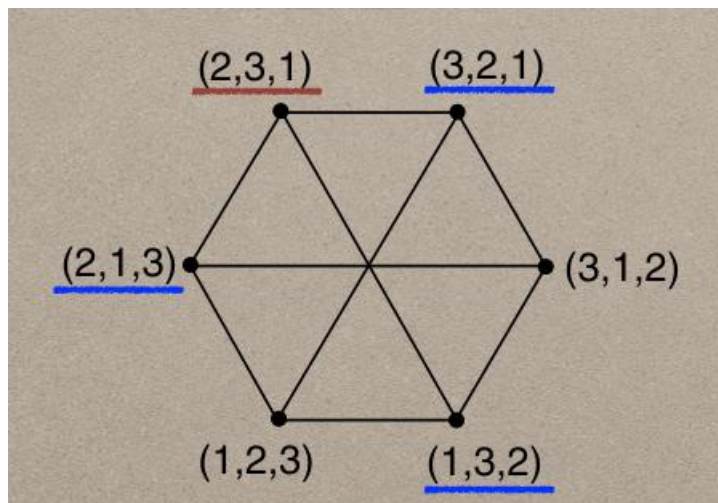


La distanza di Hamming tra X e Y è 1 (un solo bit differente). Il vicinato è costituito da tutte le stringhe ottenibili flipando un bit.

Esempio in rappresentazione intera

Con una rappresentazione basata su permutazioni di vertici, l'operatore tipico è lo swap, che scambia la posizione di due elementi.

- Il vicinato di dimensione k è formato da tutte le soluzioni ottenibili effettuando al massimo k swap.
- Per n elementi, il numero massimo di swap possibili è $\frac{n(n-1)}{2}$.

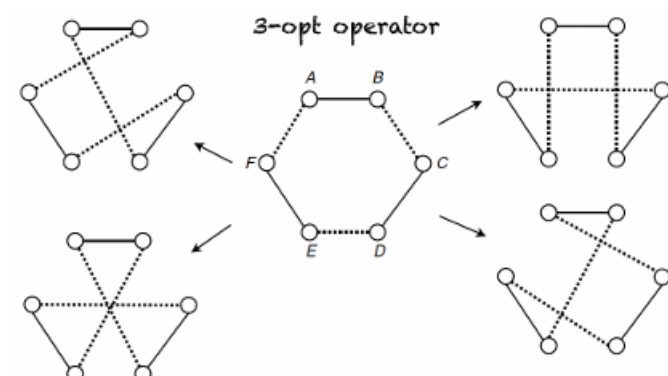


Questo è un esempio di vicinato per un problema di permutazione di dimensione 3.

K-opt operator

L'operatore K-opt consiste nel rimuovere k archi dalla soluzione corrente e sostituirli con altri k archi per ottenere una nuova soluzione.

3-opt operator: Il vicinato generato dall'operatore 3-opt è costituito da tutte le permutazioni ottenute rimuovendo due archi dalla soluzione e ricollegando i segmenti in modo diverso.

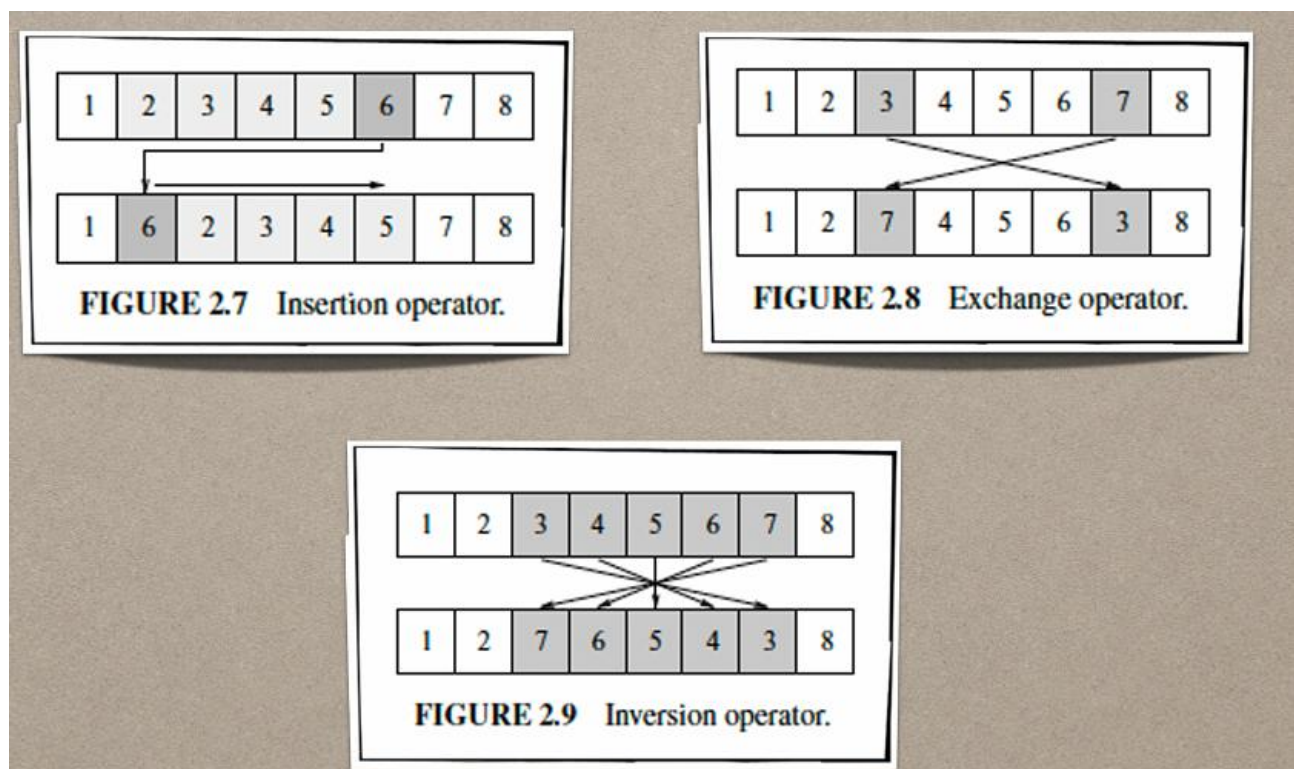


È importante sottolineare che l'**efficienza di un vicinato** è legata **non solo alla rappresentazione**, ma anche al **tipo di problema da risolvere**. Due principali categorie:

- **Problemi di scheduling (pianificazione):**
Una permutazione rappresenta una coda di priorità.
L'ordine relativo nella sequenza è importante (2-opt → località *debole*).
- **Problemi di routing (instradamento):**
L'adiacenza degli elementi è cruciale (2-opt → località *forte*). (Esempio: TSP)

NOTA: Per i problemi di scheduling, la famiglia di operatori *k-opt* non è adatta.

- **Position-based neighborhood:** modifica la posizione di un elemento (es. inserimento).
- **Order-based neighborhood:** modifica l'ordine degli elementi (es. swap o inversione di sottosequenze).



Considerazioni sulla dimensione del vicinato

- Vicinati grandi offrono maggiore possibilità di miglioramento, ma aumentano il costo computazionale.
- Vicinati piccoli sono più rapidi da esplorare, ma rischiano di portare a soluzioni subottimali.
- È necessario un compromesso tra qualità e costo computazionale.

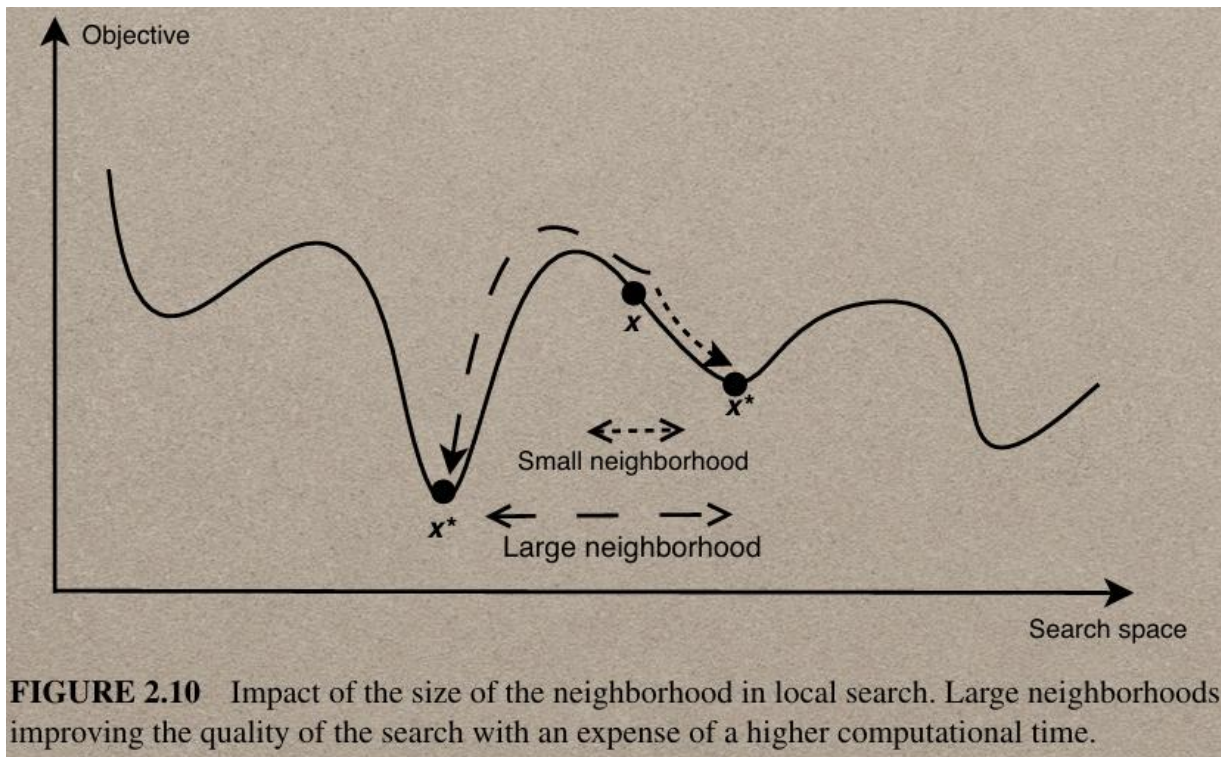
Le soluzioni peggiori non vanno sempre scartate: possono aiutare l'algoritmo a esplorare nuove regioni dello spazio di ricerca.

Compromesso: Tra dimensione/qualità del vicinato e complessità computazionale.

Progettare un vicinato ampio può migliorare la qualità delle soluzioni ottenute.

Svantaggio: Tempo computazionale aggiuntivo! La complessità della ricerca aumenta significativamente.

Obiettivo: Progettare procedure efficienti per esplorare vicinati ampi.



La dimensione del vicinato per una soluzione s è il numero di soluzioni vicine a s .

Tabu Search

Obiettivo

Minimizzare una funzione obiettivo $f(x)$, esplorando lo spazio delle soluzioni per trovare il minimo globale anziché fermarsi a un minimo locale.

Algoritmi di Local Search (Ricerca Locale):

Partono da una soluzione iniziale x e la modificano iterativamente migliorando $f(x)$.
La traiettoria di esplorazione segue una sequenza del tipo:

$$x' \rightarrow x'' \rightarrow x''' \quad \text{dove} \quad f(x') > f(x'') > f(x''')$$

Problema: Convergono spesso a un minimo locale, senza garantire l'ottimalità globale.

Introduzione alla Tabu Search

La Tabu Search (TS) estende la Local Search introducendo meccanismi per:

- **Evitare minimi locali** accettando anche soluzioni temporaneamente peggiorative.
- **Prevenire cicli** (ritorno a soluzioni già visitate) tramite una memoria strutturata (Tabu List).

Funzionamento Base

1. Soluzione Iniziale: Si parte da una soluzione x .

2. Generazione del Vicinato:

- Per ogni iterazione, si esplora un insieme di soluzioni vicine (vicinato) ottenute modificando x .
- A differenza della Local Search classica, non ci si limita a soluzioni migliorative.

3. Selezione della Nuova Soluzione:

- Si sceglie la migliore soluzione nel vicinato, anche se peggiora $f(x)$.
- Questo permette di fuggire da ottimi locali.

4. Memoria (Tabu List):

- Una lista tabù memorizza mosse o soluzioni recenti per evitare di rivisitarle.
- Aggiornata ad ogni iterazione, costituisce la memoria a breve termine.
- Impedisce all'algoritmo di ciclare tra le stesse soluzioni.

Local Search	Tabu Search
Accetta solo soluzioni migliorative	Accetta soluzioni non migliorative per esplorare meglio lo spazio.
Converge rapidamente a un minimo locale.	Può sfuggire a minimi locali grazie alla memoria e alle mosse tabù.
Nessuna memoria delle soluzioni passate.	Usa una Tabu List per evitare cicli.
Traiettoria deterministica (sempre in discesa).	Traiettoria più flessibile, con esplorazione intelligente.

Vantaggi della Tabu Search

- Scappa da ottimi locali accettando soluzioni temporaneamente peggiori.
- Riduce il rischio di cicli grazie alla Tabu List.
- Esplorazione più efficace dello spazio delle soluzioni rispetto a metodi greedy

Svantaggi

- Complessità computazionale maggiore a causa della gestione della memoria.
- Dipende dai parametri (dimensione della Tabu List, criterio di aspirazione).

Pseudocode

Algorithm 2.9 Template of tabu search algorithm.

```
 $s = s_0$  ; /* Initial solution */  
Initialize the tabu list, medium-term and long-term memories ;  
Repeat  
    Find best admissible neighbor  $s'$  ; /* non tabu or aspiration criterion holds */  
     $s = s'$  ;  
    Update tabu list, aspiration conditions, medium and long term memories ;  
    If intensification_criterion holds Then intensification ;  
    If diversification_criterion holds Then diversification ;  
Until Stopping criteria satisfied  
Output: Best solution found.
```

Problemi di progettazione specifici per una Tabu Search semplice

- **Lista Tabù:** L'obiettivo dell'uso della memoria a breve termine è impedire alla ricerca di rivisitare soluzioni già esplorate. Come accennato, memorizzare l'elenco di tutte le soluzioni visitate non è pratico per questioni di efficienza.
- **Criterio di aspirazione:** Un criterio di aspirazione comunemente usato consiste nel selezionare una mossa tabù se genera una soluzione migliore della migliore trovata finora. Un altro criterio può essere quello di accettare una mossa tabù che produca una soluzione migliore tra quelle che possiedono un determinato attributo.

Meccanismi avanzati comuni nella Tabu Search

- **Intensificazione (memoria a medio termine):** La memoria a medio termine memorizza le soluzioni d'élite (ad esempio, le migliori) trovate durante la ricerca. L'idea è dare priorità agli attributi di queste soluzioni, solitamente in modo probabilistico e ponderato.
- **Diversificazione (memoria a lungo termine):** La memoria a lungo termine registra informazioni sulle soluzioni visitate durante la ricerca. Esplora aree non correlate dello spazio delle soluzioni, ad esempio scoraggiando gli attributi delle soluzioni d'élite nelle nuove soluzioni generate, per diversificare la ricerca verso altre regioni dello spazio.

Tabu Search applicato al Sudoku

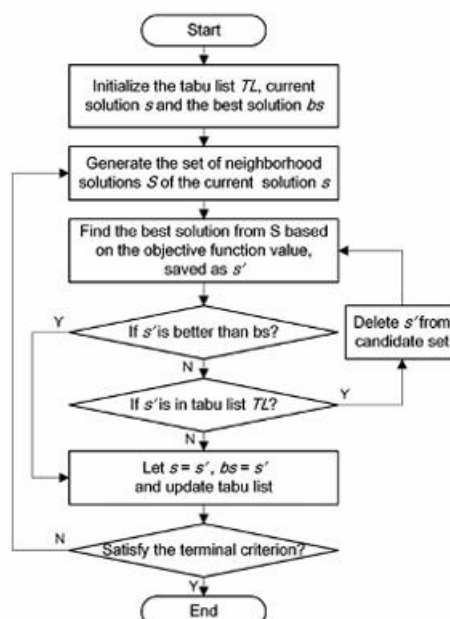
Il Sudoku è un popolare rompicapo logico basato sui numeri. La griglia è composta da una matrice di dimensioni $N \times N$, suddivisa in regioni più piccole di dimensione $n \times n$, dove $n = \sqrt{N}$.

Regole fondamentali:

1. Ogni riga deve contenere tutti i numeri da 1 a N senza ripetizioni.
2. Ogni colonna deve contenere tutti i numeri da 1 a N senza ripetizioni.
3. Ogni regione deve contenere tutti i numeri da 1 a N senza ripetizioni.

Tabu Search - Riepilogo e concetti chiave

- **Ricerca locale migliorativa:** Parte da una soluzione iniziale e cerca di migliorarla esplorando il vicinato.
- **Memoria Tabù:** Utilizza una lista tabù per evitare di tornare su soluzioni già visitate o per impedire cicli.
 - La lista tabù è una memoria a breve termine che registra le mosse o soluzioni recenti.
- **Accettazione di soluzioni subottimali:** Permette di superare minimi locali temporaneamente peggiorando la soluzione.
- **Strategie di diversificazione:** Introduce variazioni casuali o calcolate per esplorare nuove aree dello spazio delle soluzioni.



Cosa ci serve per il Tabu Search per risolvere il Sudoku?

- Soluzione Iniziale
- Funzione di Fitness
- Funzione di generazione del vicinato
- Come utilizzare la Memoria Tabù

Generazione della soluzione iniziale

- Un possibile approccio per generare una soluzione iniziale consiste nel leggere tutti i valori fissi (elementi "fixed" del problema), ovvero i numeri già presenti nella griglia.
- Per ogni sottoblocco, si distribuiscono casualmente gli elementi mancanti utilizzando i valori disponibili nel sottoblocco, senza considerare eventuali violazioni delle regole relative a righe e colonne.
- Nell'immagine, i valori in verde rappresentano gli elementi fissi, mentre i valori in bianco indicano quelli inseriti dall'utente.
- In questo modo si avranno duplicati solo lungo le righe e le colonne!

5	2	4	4	3	7	6	8	9
6	7	9	1	5	9	2	7	3
8	1	3	6	8	2	4	1	5
4	3	1	6	7	5	4	8	1
5	8	6	1	8	9	9	3	2
7	9	2	2	3	4	7	6	5
2	8	9	7	1	5	8	4	2
6	4	5	2	9	3	9	1	3
3	1	7	8	6	4	7	5	6

Fitness Function

- Come funzione di fitness, possiamo sfruttare il vincolo che nei sottoblocchi non possono esserci duplicati e concentrare il calcolo delle violazioni solo su righe e colonne. Nello specifico, si calcola il numero totale di violazioni, ovvero i duplicati presenti in ciascuna riga e in ciascuna colonna.
- La funzione di costo è quindi definita come la somma complessiva delle violazioni:
 - Somma delle violazioni per ogni riga
 - Somma delle violazioni per ogni colonna
- Nell'immagine mostrata, il costo della soluzione proposta è pari a 38.
- Il problema risulta quindi essere di minimizzazione.

1	5	2	4	4	3	7	6	8	9
2	6	9	7	1	5	9	2	7	3
2	8	1	3	6	8	2	4	1	5
2	4	3	1	6	7	5	4	8	1
2	5	8	6	1	8	9	9	3	2
2	7	9	2	2	3	4	7	6	5
2	2	8	9	7	1	5	8	4	2
2	6	4	5	2	9	3	9	1	3
2	3	1	7	8	6	4	7	5	6
	2	3	1	3	2	3	3	2	3

Generazione del vicinato

La generazione delle soluzioni vicine avviene attraverso una modifica controllata della griglia corrente, seguendo questa procedura:

1. **Selezione casuale di un blocco** Viene scelto in modo casuale uno dei sottoblocchi 3×3 della griglia sudoku.
2. **Identificazione degli elementi modificabili** All'interno del blocco selezionato, vengono individuate le celle con valori non fissati (quelle non marcate come "fixed").
3. **Scambio casuale** di valori Si selezionano casualmente due celle modificabili all'interno dello stesso blocco e si scambiano i loro valori.

Questa operazione produce una nuova configurazione della griglia, che rappresenta una soluzione vicina a quella corrente.

Local Search

Come si costruisce un vicinato? L'idea è individuare un insieme di soluzioni vicine a quella attuale (intorno) e selezionare la migliore tra queste, in modo da sostituire quella corrente con una potenzialmente migliore.

La dimensione del vicinato gioca un ruolo cruciale:

- Un **vicinato ristretto** consente di trovare rapidamente un buon ottimo locale, ma rischia di trascurare soluzioni migliori.
- Un **vicinato ampio** permette di esplorare soluzioni di qualità superiore, ma il costo computazionale cresce rapidamente con il numero di candidati da valutare.

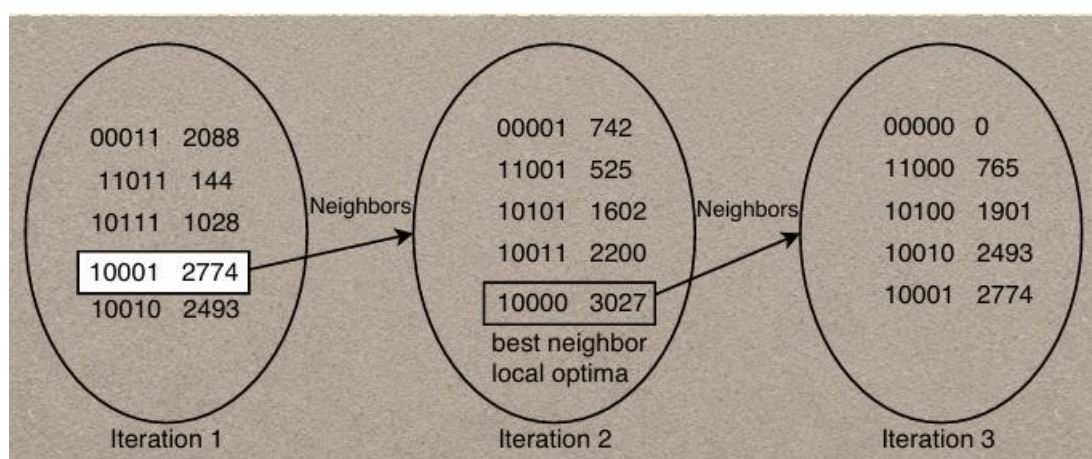
È quindi necessario trovare un buon compromesso nella dimensione del vicinato: né troppo piccola, né troppo grande. Algoritmi come la Tabu Search sono progettati per bilanciare efficacemente questo trade-off, guidando la ricerca verso soluzioni migliori senza esplorare inutilmente spazi troppo vasti.

Algoritmo di Ricerca Locale

La ricerca locale è un metodo semplice e classico. A partire da una soluzione iniziale, il procedimento consiste nel sostituire iterativamente la soluzione corrente con una tra le sue vicine che migliora la funzione obiettivo.

Il processo termina quando nessuna delle soluzioni nel vicinato offre un miglioramento rispetto a quella corrente.

Poiché la soluzione iniziale è generalmente scelta in modo casuale, la qualità dell'ottimo locale ottenuto dipende fortemente da questa scelta. Una cattiva soluzione iniziale può portare a risultati subottimali, mentre una buona può guidare rapidamente verso un ottimo locale soddisfacente.



Pseudocodice

Algorithm 2.2 Template of a local search algorithm.

```
 $s = s_0$  ; /* Generate an initial solution  $s_0$  */  
While not Termination_Criterion Do  
    Generate ( $N(s)$ ) ; /* Generation of candidate neighbors */  
    If there is no better neighbor Then Stop ;  
     $s = s'$  ; /* Select a better neighbor  $s' \in N(s)$  */  
Endwhile  
Output Final solution found (local optima).
```

Le varianti della Local Search (LS) possono essere distinte in base a due principali caratteristiche:

1. Ordine di generazione delle soluzioni nel vicinato:

- **Deterministico:** le soluzioni vicine vengono generate seguendo un ordine fisso e predefinito.
- **Stocastico:** le soluzioni vengono generate in modo casuale o secondo una certa probabilità.

2. Strategia di selezione delle soluzioni nel vicinato:

- La scelta della soluzione da adottare tra quelle generate può seguire strategie diverse.

Selezione del Vicinato

Durante la ricerca locale, una volta generato il vicinato, è necessario scegliere quale soluzione adottare per la prossima iterazione. Le principali strategie di selezione sono:

Best Improvement (Steepest Descent)

Questa strategia consiste nel valutare tutte le soluzioni del vicinato e selezionare quella che migliora maggiormente la funzione obiettivo.

- L'esplorazione del vicinato è esaustiva e completamente deterministica: tutte le mosse possibili vengono provate.
- Permette di trovare rapidamente un ottimo locale in poche iterazioni.
- Tuttavia, può risultare computazionalmente costosa, soprattutto se il vicinato è molto grande.

First Improvement

In questo approccio si esplora il vicinato secondo un ordine prestabilito e si accetta la prima soluzione che migliora la funzione obiettivo rispetto a quella attuale.

- L'esplorazione è parziale, ma comunque deterministica.
- Se non si trova alcun miglioramento, l'intero vicinato viene comunque valutato (caso peggiore).
- Questa strategia può richiedere più iterazioni per raggiungere un ottimo locale, ma consente una maggiore varietà nei percorsi esplorati.
- Offre una maggiore diversificazione nella ricerca: consente di esplorare più zone del landscape, aprendo la strada a soluzioni potenzialmente migliori in direzioni diverse.

Random Selection

In questa strategia viene selezionata casualmente una delle soluzioni migliorative presenti nel vicinato.

- Richiede comunque l'esplorazione dell'intero landscape per individuare le soluzioni migliorative da cui scegliere.
- Introdurre casualità consente di evitare stagnazioni in ottimi locali e di migliorare la diversità della ricerca.

Considerazioni sulla Dimensione del Vicinato

La dimensione del vicinato influisce direttamente sull'efficacia e sull'efficienza di ciascuna strategia:

- Un vicinato più grande consente di identificare un numero maggiore di ottimi locali e potenziali soluzioni di alta qualità.
- Tuttavia, l'aumento del numero di soluzioni da considerare rende più complessa e onerosa la manipolazione dei risultati, in particolare per strategie come il Best Improvement.
- Con strategie come il First Improvement o la Random Selection, è possibile gestire vicinati ampi senza dover necessariamente esplorare tutto il landscape, ottenendo comunque una buona varietà di scenari e direzioni esplorative.

Iterated Local Search

L'Iterated Local Search (ILS) nasce con l'obiettivo di superare i limiti dei classici algoritmi di ricerca locale, come la Simple Local Search o la Tabu Search, che tendono a rimanere intrappolati in ottimi locali. Questi algoritmi, infatti, sostituiscono la soluzione corrente solo se trovano una soluzione migliore, il che impedisce loro di esplorare efficacemente altre regioni dello spazio di ricerca.

ILS introduce una componente fondamentale: la perturbazione controllata della soluzione attuale, permettendo all'algoritmo di esplorare nuove aree del landscape e potenzialmente individuare soluzioni globalmente migliori. A differenza delle strategie puramente deterministiche, l'ILS combina casualità e ottimizzazione locale, sfruttando i punti di forza di entrambi gli approcci.

Obiettivo

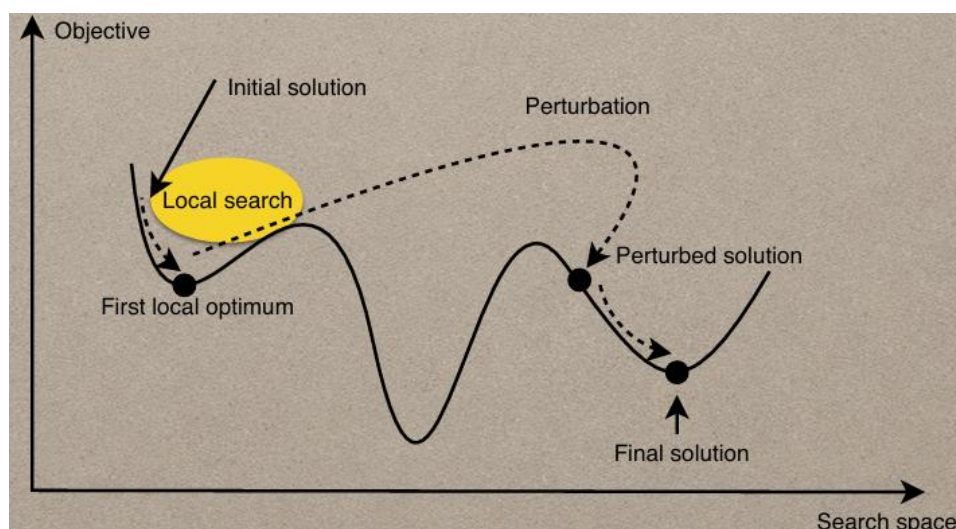
Non vincolarsi a scegliere solo soluzioni migliorative, ma accettare anche soluzioni peggiori per uscire dagli ottimi locali e raggiungere soluzioni di qualità superiore.

Struttura dell'ILS

L'Iterated Local Search si basa su tre componenti fondamentali:

1. Local Search su una soluzione iniziale

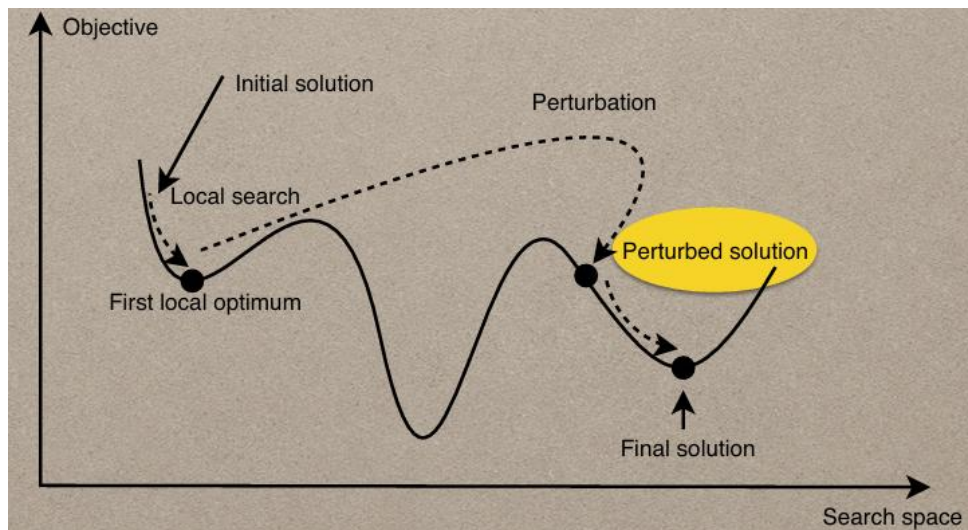
Si parte da una soluzione iniziale scelta casualmente, su cui si applica un algoritmo di ricerca locale (deterministico o stocastico). Questo porta a un primo ottimo locale.



2. Perturbazione della soluzione

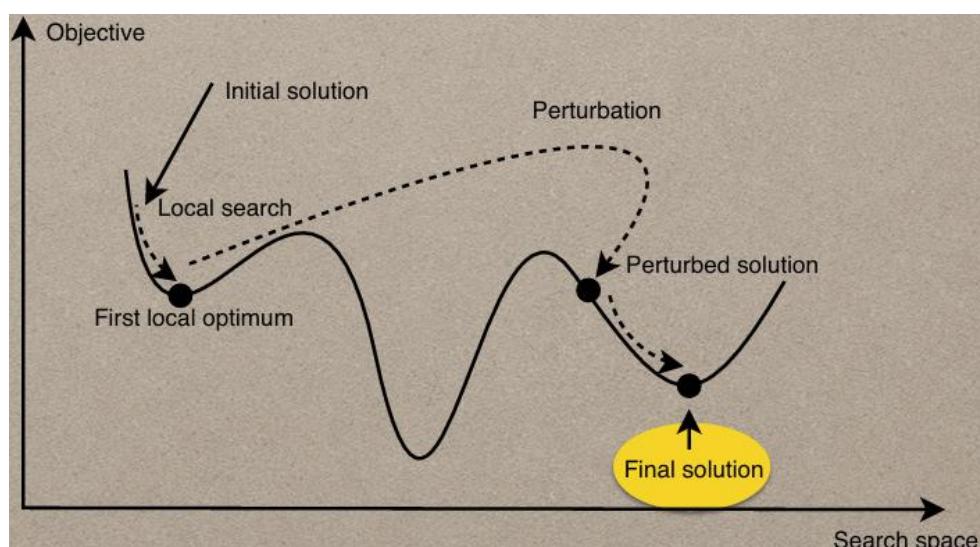
La soluzione ottimizzata viene poi perturbata attraverso una modifica significativa e stocastica.

- L'obiettivo è quello di uscire dalla regione dell'ottimo locale attuale e accedere a nuove aree del landscape.
- La perturbazione mantiene intatta una parte della soluzione ma alterando fortemente un'altra, permettendo di "saltare" in un altro bacino di attrazione.



3. Nuova Local Search

Sulla soluzione perturbata viene nuovamente applicata una Local Search, per raffinare in modo deterministico la qualità della nuova soluzione.



Criterio di Accettazione (Acceptance Criterion)

Definisce le condizioni in base alle quali la nuova soluzione ottimizzata viene accettata come corrente. Questo criterio può variare:

- Accettare solo se la nuova soluzione è migliore
- Accettare con una certa probabilità anche soluzioni peggiori (per evitare stagnazione)
- Oppure strategie ibride

Caratteristiche e Vantaggi

- Superamento degli ottimi locali: accettando soluzioni peggiori, ILS evita la trappola dei minimi locali.
- Combinazione efficace di casualità e determinismo:
 - La casualità (scelta iniziale e perturbazione) consente un'esplorazione più ampia dello spazio delle soluzioni.
 - L'ottimizzazione deterministica della ricerca locale affina le soluzioni trovate, migliorandone la qualità.
- Flessibilità: all'interno dell'ILS si possono impiegare diverse tecniche di local search (simple descent, tabu search, simulated annealing), trattandole come black box.

Pseudocodice

Algorithm 2.10 Template of the iterated local search algorithm.

```
 $s_*$  = local search( $s_0$ ) ; /* Apply a given local search algorithm */  
Repeat  
   $s' = \text{Perturb}(s_*, \text{search history})$  ; /* Perturb the obtained local optima */  
   $s'_* = \text{Local search}(s')$  ; /* Apply local search on the perturbed solution */  
   $s_* = \text{Accept}(s_*, s'_*, \text{search memory})$  ; /* Accepting criteria */  
Until Stopping criteria  
Output: Best solution found.
```

Metodi di perturbazione

La lunghezza di una perturbazione può essere correlata al vicinato associato alla codifica della soluzione oppure al numero di componenti modificate.

- Una **perturbazione troppo piccola** può portare a cicli nella ricerca, senza ottenere alcun miglioramento.
La probabilità di esplorare altri bacini di attrazione risulta molto bassa.
- Una **perturbazione troppo grande**, invece, può cancellare le informazioni utili accumulate durante la ricerca (search memory) e perdere le buone caratteristiche della soluzione ottimizzata localmente.

Criteri di Accettazione

Il ruolo del criterio di accettazione, combinato con il metodo di perturbazione, è quello di controllare il classico compromesso tra i compiti di intensificazione e diversificazione. La soluzione estrema in termini di intensificazione è accettare solo soluzioni migliorative rispetto alla funzione obiettivo (selezione forte). La soluzione estrema in termini di diversificazione è accettare qualsiasi soluzione senza considerarne la qualità (selezione debole). Possono essere applicati molti criteri di accettazione che bilanciano questi due obiettivi:

- **Criteri di accettazione probabilistici:** In letteratura si trovano molti criteri di accettazione probabilistici. Ad esempio, la distribuzione di Boltzmann del simulated annealing. In questo caso, è necessario definire un programma di raffreddamento.
- **Criteri di accettazione deterministici:** Alcuni criteri di accettazione deterministici possono essere ispirati agli algoritmi di threshold accepting e agli algoritmi correlati, come il great deluge e il record-to-record.

Perché un problema è complesso?

Un problema viene considerato complesso quando:

- La dimensione dei dati da elaborare è molto elevata.
- L'esplorazione dello spazio delle soluzioni richiede troppo tempo.
- L'uso di algoritmi deterministici diventa impraticabile o inefficiente.
- Sono presenti incertezze, cambiamenti dinamici e vincoli temporali che rendono difficile modellare e risolvere il problema in modo esatto.

In questi casi, si preferiscono metodi euristici o metaeuristici, ispirati spesso a fenomeni naturali, che cercano soluzioni approssimate ma accettabili in tempi ragionevoli.

Algoritmi basati sulla popolazione

A differenza degli approcci tradizionali, che migliorano una singola soluzione alla volta, questi algoritmi operano su un insieme di soluzioni, detto popolazione, che evolve nel tempo.

Questa strategia permette di:

- Esplorare simultaneamente più regioni dello spazio delle soluzioni.
- Evitare di rimanere bloccati in ottimi locali.
- Favorire la diversità, migliorando la qualità delle soluzioni nel lungo termine.

Costruire meccanismi ispirati alla natura

Ispirarsi alla natura significa osservare come i sistemi naturali affrontano e risolvono problemi complessi, come:

- l'evoluzione delle specie,
- il comportamento degli insetti sociali,
- il funzionamento del sistema immunitario.

Teoria dell'evoluzione

La selezione naturale favorisce solo gli individui più adatti al contesto ambientale. Il concetto di “più forte” è relativo e dipende dalle condizioni.

Esempio: il leone è dominante nella savana, ma non sopravviverebbe al Polo Nord. Ciò che conta è la capacità di adattamento.

L'evoluzione naturale porta a generazioni più robuste e performanti.

Quattro concetti chiave

- **Adattamento:** capacità di rispondere ai cambiamenti ambientali.
- **Crossover:** combinazione del patrimonio genetico dei genitori per produrre nuova prole.
- **Mutazione:** variazioni casuali che introducono diversità.
- **Evoluzione:** processo iterativo che migliora la popolazione nel tempo.

Tipologie di algoritmi ispirati alla natura

- **Algoritmi genetici:** simulano l'evoluzione biologica (selezione, crossover, mutazione).
- **Sistemi immunitari artificiali:** imitano il sistema immunitario, memorizzano eventi passati e rispondono rapidamente a minacce già note (es. concetti applicabili alla cybersecurity).

Swarm Intelligence

Si tratta di intelligenza collettiva osservabile in sistemi naturali come stormi, banchi di pesci o sciami d'insetti.

Ogni individuo segue regole semplici, ma il comportamento collettivo è complesso e coordinato.

Esempi di algoritmi

- **Particle Swarm Optimization (PSO):** simula il comportamento di particelle che si muovono nello spazio delle soluzioni.
- **Ant Colony Optimization (ACO):** si basa sul comportamento delle formiche nella ricerca di cibo.

Colonie di formiche (Ant Colony Optimization)

Le formiche:

- comunicano tramite feromoni,
- trovano il cammino minimo tra nido e cibo,
- si adattano ai cambiamenti in modo efficiente.

Maggiore è l'intensità del feromone su un percorso, maggiore è la probabilità che venga seguito → ottimizzazione collettiva.

Evoluzione naturale

- Ogni individuo appartiene a una popolazione.
- Agisce localmente ma in un contesto condiviso.
- Le coppie generano nuova prole, che eredita tratti genetici (con possibili mutazioni).
- L'intero processo tende a migliorare la qualità media della popolazione.

Algoritmo genetico

1. Generazione iniziale della popolazione casuale.
2. Valutazione delle soluzioni tramite una fitness function.
3. Selezione dei migliori individui.
4. Crossover tra individui selezionati per generare figli.
5. Mutazione di alcuni elementi della prole.
6. Sostituzione della vecchia popolazione con la nuova.

Si ripete fino al raggiungimento:

- di una soluzione accettabile, oppure
- del numero massimo di iterazioni.

Pseudocodice

1. Generate the initial **population** $P(0)$ at random, and set $i \leftarrow 0$;
2. REPEAT
 - (a) Evaluate the fitness of each individual in $P(i)$;
 - (b) **Select** parents from $P(i)$ based on their fitness in $P(i)$;
 - (c) **Generate** offspring from the parents using *crossover* and *mutation* to form $P(i + 1)$;
 - (d) $i \leftarrow i + 1$;
3. UNTIL halting criteria are satisfied

Se si selezionano sempre solo i migliori, si rischia di **bloccarsi** in un ottimo locale. Una certa variabilità genetica aumenta le probabilità di trovare l'ottimo globale.

Come funziona

Per affrontare un problema mediante un algoritmo genetico, si inizia **generando casualmente** un insieme iniziale di soluzioni candidate, ciascuna rappresentata come una stringa di lunghezza fissa. Ogni soluzione viene poi valutata attraverso una **funzione di fitness**, che misura quanto essa sia adatta a risolvere il problema. Le soluzioni con i punteggi migliori vengono selezionate e utilizzate per generare nuove soluzioni, mediante operazioni come il **crossover** e la **mutazione**. Questo processo di valutazione e generazione viene ripetuto ciclicamente fino a quando non si ottiene una soluzione considerata soddisfacente oppure fino al raggiungimento di un numero massimo di iterazioni, detto anche generazioni.

Operatori dell'algoritmo genetico

La **riproduzione** è solitamente il primo operatore applicato alla popolazione. In questa fase, vengono selezionati alcuni cromosomi dalla popolazione dei genitori per effettuare il crossover e generare nuova prole. Questo processo si basa sulla teoria dell'evoluzione di Darwin, nota come "sopravvivenza del più adatto", motivo per cui l'operatore di riproduzione è anche chiamato **operatore di selezione**.

Successivamente, dopo la fase di riproduzione, la popolazione viene arricchita con **individui di qualità migliore**. Tuttavia, questo passaggio tende a produrre semplicemente delle copie (cloni) delle soluzioni migliori, senza introdurre vera diversità. Per ovviare a questo limite, si applica **l'operatore di crossover** al cosiddetto mating pool (gruppo di accoppiamento), nella speranza che la combinazione delle stringhe genitoriali generi soluzioni migliori rispetto a quelle esistenti.

Infine, dopo il crossover, le stringhe ottenute vengono sottoposte **all'operatore di mutazione**, che introduce variazioni casuali nei cromosomi. Nel caso più semplice, la mutazione consiste nell'inversione di un singolo bit, ovvero trasformare uno 0 in 1 o viceversa. Questo meccanismo serve a mantenere la diversità genetica all'interno della popolazione ed evita che l'algoritmo si blocchi in ottimi locali.

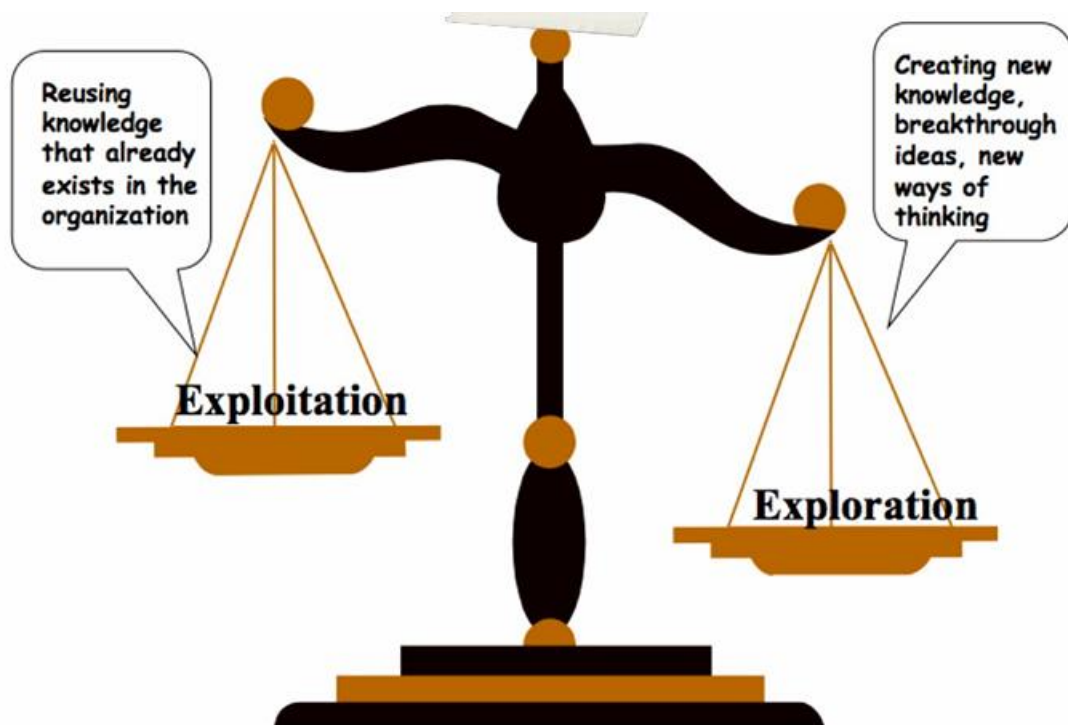
Exploitation ed Exploration

Nel contesto degli Algoritmi Genetici, exploitation ed exploration rappresentano due forze complementari che guidano il processo evolutivo.

- **Exploitation** consiste nello sfruttare le conoscenze già acquisite per affinare le soluzioni esistenti. Mira a concentrare la ricerca nelle regioni promettenti dello spazio delle soluzioni, migliorando progressivamente la qualità delle soluzioni attuali.
- **Exploration**, invece, ha lo scopo di esplorare nuove aree dello spazio delle soluzioni, favorendo la scoperta di nuove informazioni e la diversificazione della popolazione. Serve a evitare la convergenza prematura verso ottimi locali.

Gli operatori agiscono in modo diverso in base a queste due dinamiche:

- L'operatore di **selezione** è tipicamente associato alla **exploitation**, poiché favorisce la riproduzione degli individui con fitness più elevato, permettendo al GA di approfondire le aree già identificate come promettenti.
- Gli operatori di variazione (come **mutazione** e alcuni tipi di **crossover** perturbativi) sono invece responsabili della exploration, poiché introducono diversità nella popolazione, generando nuove soluzioni potenzialmente migliori.



Operatore di Selezione

La riproduzione è il processo mediante il quale vengono scelti i genitori per generare nuovi individui. Questo compito è svolto dall'operatore di selezione, il cui obiettivo è **individuare le soluzioni migliori** (o comunque promettenti) all'interno della popolazione corrente. Selezionare semplicemente i migliori individui è una strategia intuitiva ma spesso subottimale, in quanto può portare a una perdita di diversità genetica. Esistono diverse strategie di selezione che bilanciano qualità ed eterogeneità:

- **Selezione proporzionale alla fitness:** ogni individuo viene selezionato con una probabilità proporzionale al proprio valore di fitness. Se f_i è la fitness dell'individuo i , allora la sua probabilità di selezione è:

$$P_i = \frac{f_i}{\sum f_i}$$

In questo modo, gli individui con fitness più elevata hanno una maggiore probabilità di essere scelti, ma anche gli altri conservano una certa possibilità.

- **Selezione a torneo:** si sceglie casualmente un sottoinsieme di k individui (dove k è un parametro fissato) dalla popolazione totale. Tra questi, viene selezionato il migliore secondo la fitness. Aumentare il valore di k rende la selezione più competitiva, migliorando l'efficacia dell'algoritmo ma riducendo la diversità.
- **Roulette Wheel Selection:** immaginando una roulette suddivisa in settori proporzionali ai valori di fitness, ogni individuo occupa uno spazio proporzionale alla propria fitness. Supponiamo di avere 4 individui con fitness 7, 12, 21 e 15. La somma totale è 55. I settori sono:
 - da 1 a 7: individuo con fitness 7
 - da 8 a 19: individuo con fitness 12
 - da 20 a 40: individuo con fitness 21
 - da 41 a 55: individuo con fitness 15

Generando un numero casuale tra 1 e 55 si effettua la selezione. Anche qui, individui meno performanti mantengono una probabilità di essere scelti, il che aiuta a preservare la diversità genetica.

Una differenza fondamentale tra questi approcci è che la selezione a torneo introduce un parametro k che consente di controllare direttamente la pressione selettiva: maggiore è k , più l'algoritmo tende a privilegiare i migliori, ma a scapito dell'eterogeneità.

Selezione degli Individui per la Generazione Successiva

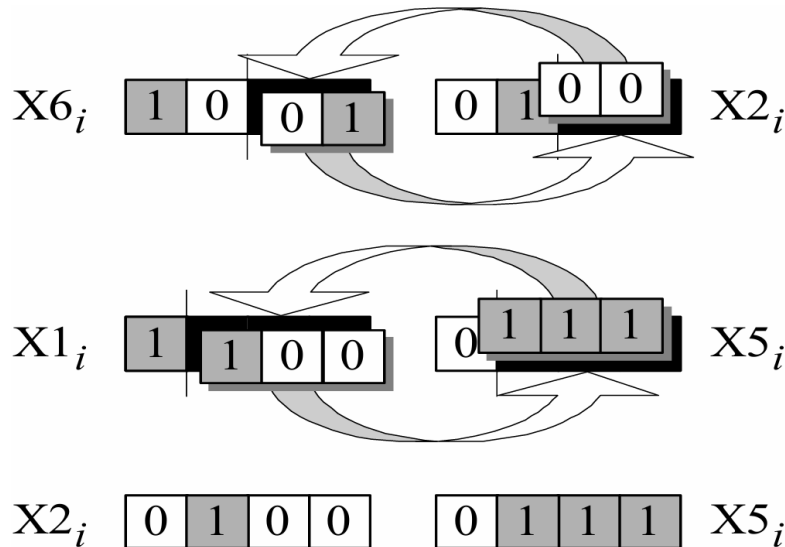
Per costruire la popolazione del passo successivo si può agire in due modi, a seconda del metodo di selezione adottato:

- (μ, λ) – *Selection*: si selezionano gli individui esclusivamente dalla popolazione dei discendenti (λ). I genitori della generazione corrente non partecipano alla selezione successiva. Questo approccio favorisce l'introduzione di variabilità, ma può sacrificare buone soluzioni già trovate.
- $(\mu + \lambda)$ – *Selection*: si effettua la selezione combinando la popolazione iniziale (μ) con quella dei discendenti (λ). Si scelgono i migliori individui dall'unione delle due. Questo metodo tende a essere più conservativo e favorisce la conservazione delle soluzioni migliori già esistenti.

Anche in questa fase si possono riutilizzare gli stessi metodi di selezione usati nella riproduzione (proporzionale, torneo, roulette), al fine di mantenere un buon livello di eterogeneità nella popolazione. L'equilibrio tra esplorazione e sfruttamento (exploration vs. exploitation) resta un aspetto fondamentale per la buona riuscita dell'algoritmo genetico.

Crossover

Operazione che combina due genitori per generare nuovi individui.



One Point Crossover

Si sceglie un punto di taglio comune nelle stringhe:

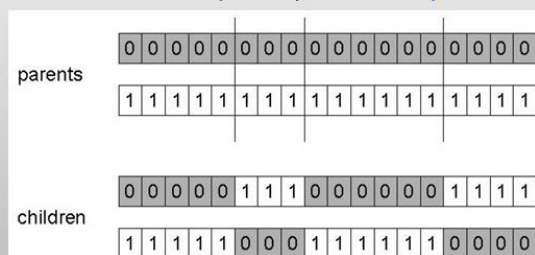
- Genitore A: 11001 | 001
- Genitore B: 00100 | 111

Si ottengono:

- Figlio 1: 11001 + 111 = 11001111
- Figlio 2: 00100 + 001 = 00100001

È possibile usare più punti di taglio.

- Choose n random crossover points
- Split along those points
- Glue parts, alternating between parents
- Generalisation of 1 point (still some positional bias)



Uniform Crossover

L'Uniform Crossover combina due cromosomi genitori bit per bit usando una maschera casuale della stessa lunghezza dei genitori:

- per ogni posizione i la maschera contiene 0 oppure 1;
- se la maschera vale 0, il Figlio C prende il bit da A e il Figlio D prende il bit da B;
- se la maschera vale 1, il Figlio C prende il bit da B e il Figlio D prende il bit da A.

Questo meccanismo introduce la massima mescolanza possibile mantenendo la lunghezza fissa dei cromosomi e produce due figli complementari.

Esempio

- Genitore A: **1 0 1 1 1 0 0 0**
- Genitore B: **0 0 1 1 0 1 1 0**
- Maschera: **0 1 0 1 1 0 0 1** (generata casualmente)

Figli risultanti

- Figlio C: **1 0 1 1 0 0 0 0**
- Figlio D: **0 0 1 1 1 1 1 0**

La maschera casuale garantisce che **ogni bit abbia la stessa probabilità di provenire da ciascun genitore**, aumentando la diversità genetica tra le soluzioni generate. La presenza simultanea dei due figli (C e D) preserva tutta l'informazione dei genitori, distribuendola in modo differente e riducendo il rischio di perdita di geni potenzialmente utili nel processo evolutivo.

Crossover



Mutazione

La mutazione è un operatore fondamentale negli algoritmi genetici, anche se viene applicato con una probabilità molto bassa, solitamente compresa tra 0.001 e 0.01. Il suo ruolo principale è quello di evitare che l'algoritmo si blocchi in un ottimo locale, cioè in una soluzione che appare buona ma non è la migliore possibile.

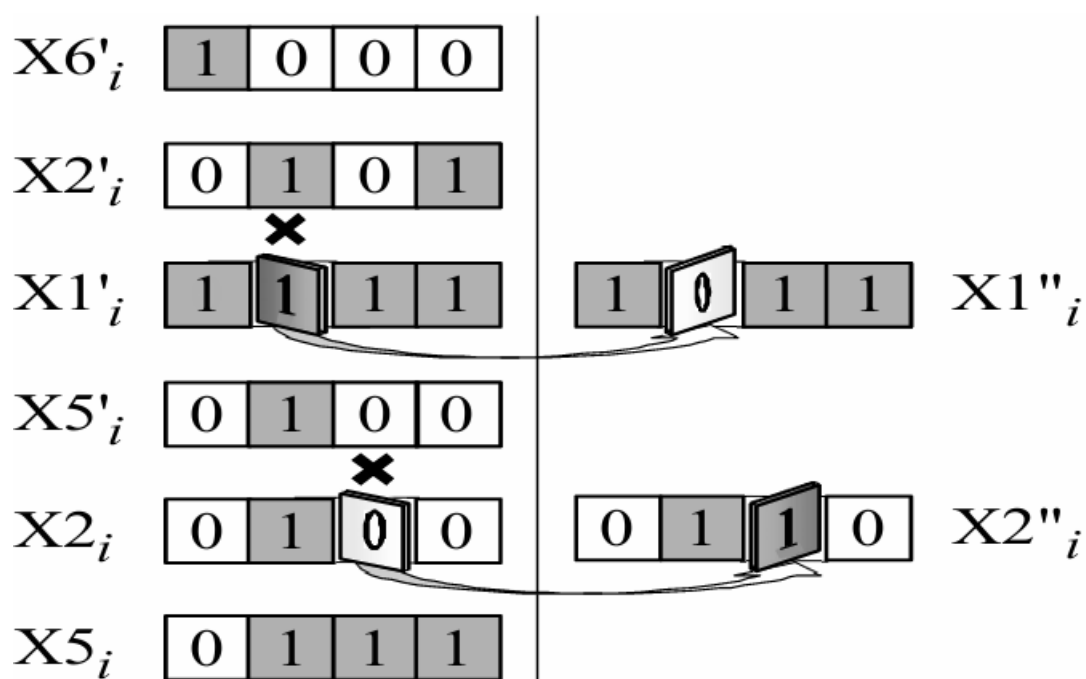
Mutare significa modificare un gene (cioè un **singolo elemento**) della soluzione corrente. Ad esempio, in una rappresentazione binaria, **flippare un bit** significa cambiare un 0 in 1 o viceversa. La mutazione avviene in modo casuale e consente di introdurre nuova diversità nella popolazione, migliorando la capacità dell'algoritmo di esplorare lo spazio delle soluzioni.

La mutazione agisce come un operatore di background, garantendo che l'esplorazione dello spazio delle soluzioni non si fermi prematuramente. Anche se raramente applicata, è essenziale per la robustezza dell'intero processo evolutivo.

Classici operatori di mutazione / perturbazione

A seconda del tipo di rappresentazione della soluzione, vengono adottate diverse strategie di mutazione:

- **Bit Flip:** Utilizzato nelle rappresentazioni binare, consiste nel selezionare casualmente un bit della soluzione e invertirlo: $0 \rightarrow 1$ oppure $1 \rightarrow 0$.



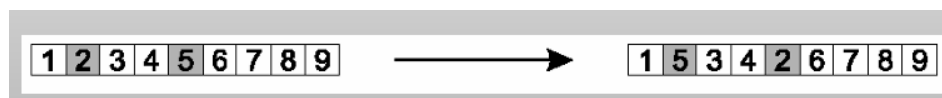
- **Insert:** Utilizzato in rappresentazioni come permutazioni o sequenze. Si selezionano due elementi casuali della soluzione e si sposta il secondo elemento accanto al primo, mantenendo l'ordine relativo.

Funzionamento

- **Seleziona due geni (alleli) a caso** nella soluzione. Esempio: data la soluzione $S = [1, 2, 3, 4, 5, 6]$, scegliamo le posizioni 2 e 5, cioè i valori 2 e 5.
- **Sposta il secondo gene scelto subito dopo il primo.** Gli elementi intermedi vengono **traslati** per lasciare spazio. Dall'esempio: spostiamo il 5 dopo il 2, ottenendo: $S = [1, 2, 5, 3, 4, 6]$
- **Preserva quasi tutto l'ordine originale e mantiene in parte le relazioni di adiacenza**, rendendolo meno distruttivo rispetto ad altri operatori come l'inversione o lo swap.



- **Swap:** Si selezionano due posizioni casuali all'interno della soluzione e si scambiano i valori tra di loro. Ad esempio: $A = [1, 2, 3, 4] \rightarrow [1, 4, 3, 2]$



- **Inversione:** Utilizzata nelle rappresentazioni a permutazione. Consiste nel selezionare due geni (alleli) casuali all'interno della soluzione e invertire la sottostringa compresa tra di essi.

Funzionamento

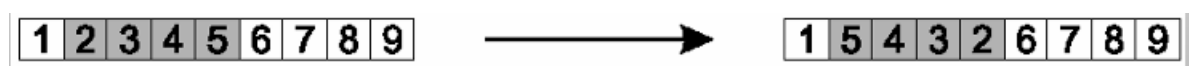
$S = [1, 2, 3, 4, 5, 6, 7]$

Indici scelti: 2 e 5 $\rightarrow$ sottostringa: $[3, 4, 5, 6]$

Dopo l'inversione: $S' = [1, 2, 6, 5, 4, 3, 7]$

Proprietà:

- Preserva gran parte delle adiacenze (rompe solo 2 legami).
- Modifica l'ordine interno della sequenza, introducendo variabilità.
- Utile per uscire da ottimi locali senza distruggere completamente la struttura.



Esempio di Applicazione di un Algoritmo Evolutivo

Obiettivo

Massimizzare la funzione:

$$f(x) = x^2 \quad \text{con } x \in [0,31]$$

Rappresentazione

Usiamo 5 bit per rappresentare gli interi da 0 a 31. Ogni individuo è quindi una stringa binaria (cromosoma)

Inizializzazione

Generiamo casualmente la **popolazione iniziale** di 4 individui:

- 01101 → 13
- 11000 → 24
- 01000 → 8
- 10011 → 19

Valutazione della Fitness

Decodifichiamo gli individui e applichiamo $f(x) = x^2$:

- $f(13)=169$
- $f(24)=576$
- $f(8)=64$
- $f(19)=361$

Totale fitness: $169 + 576 + 64 + 361 = 1170$

Selezione (Roulette Wheel)

Calcoliamo le probabilità di selezione:

- $p1 = \frac{169}{1170} \approx 0.14$
- $p2 = \frac{576}{1170} \approx 0.49$
- $p3 = \frac{64}{1170} \approx 0.05$
- $p4 = \frac{361}{1170} \approx 0.31$

Supponiamo di selezionare le coppie:

- 01101 (13) e 11000 (24)
- 10011 (19) e 11000 (24)

Crossover (One-Point)

Eseguiamo crossover a singolo punto casuale:

- Crossover tra **01101** e **11000**, punto a **4**:
 - 0110|1 + 1100|0 → 01100 e 11001
- Crossover tra **10011** e **11000**, punto a **2**:
 - 10|011 + 11|000 → 10000 e 11011

Popolazione intermedia:

- 01100, 11001, 10000, 11011

Mutazione

Applichiamo **bit flip** con bassa probabilità (es. 1%). Supponiamo che un bit venga mutato:

- 01100 → 01101 (bit 5 da 0 a 1)

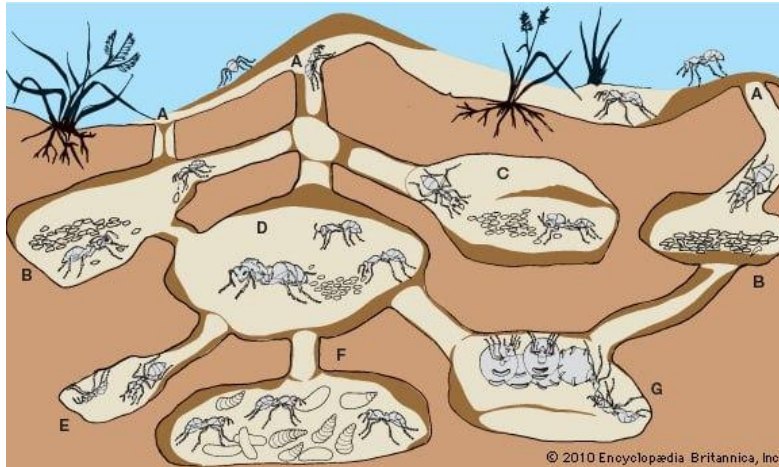
Nuova popolazione P(1):

01101, 11001, **00000**, 11011

Ripeti

Ritorna al punto 2 fino a soddisfare una condizione di arresto (es. numero massimo di generazioni o raggiungimento di una fitness target).

Ant Colony Optimization



Dalle formiche reali a quelle artificiali

Le colonie di formiche, così come più in generale le **società di insetti sociali**, rappresentano **sistemi distribuiti** complessi in cui emerge un'organizzazione altamente strutturata, nonostante la semplicità comportamentale degli individui che le compongono. Questa organizzazione collettiva consente al sistema di svolgere compiti estremamente articolati e di affrontare problematiche la cui risoluzione eccede di gran lunga le capacità del singolo. In tal senso, il comportamento collettivo non è il risultato di una supervisione centralizzata, ma piuttosto l'esito di interazioni locali, ripetitive e autonome, che danno origine a **scemi comportamentali globali**.

È proprio a partire da questa osservazione che si sviluppa il campo degli "**ant algorithms**", ovvero quell'ambito della ricerca in intelligenza artificiale e ottimizzazione che trae ispirazione dal comportamento delle formiche reali per la progettazione di algoritmi computazionali innovativi orientati alla risoluzione di problemi di Ottimizzazione Discreta.

Uno degli aspetti più affascinanti di questi sistemi è il loro funzionamento secondo **principi auto-organizzativi** (self-organizing principles). Questi principi consentono di ottenere un comportamento coordinato altamente efficace tra agenti artificiali, i quali, operando in parallelo e seguendo semplici regole locali, riescono a risolvere problemi computazionali complessi.

Il meccanismo principale attraverso cui si realizza questa coordinazione è la **stigmergia**, ovvero una forma di comunicazione indiretta, mediata da modificazioni dell'ambiente. In termini biologici, si tratta dei **feromoni** che le formiche depositano durante il tragitto tra la fonte di cibo e il nido: tali tracce chimiche non contengono istruzioni esplicite, ma influenzano profondamente le scelte delle formiche successive.

Il comportamento di **foraging** (si riferisce al modo in cui gli animali cercano e trovano cibo) di molte specie di formiche è interamente basato su questo meccanismo. Camminando lungo un cammino, una formica rilascia feromoni al suolo, creando in questo modo un **pheromone trail**. Le altre formiche, percependo tali tracce, tendono **probabilisticamente** a scegliere i percorsi in cui la concentrazione di feromone è maggiore. In tal modo, un percorso più breve (e dunque attraversabile più frequentemente) viene rinforzato più rapidamente, generando un **feedback positivo** che porta la colonia a convergere verso la soluzione più efficiente.

A sostegno empirico di questo modello, si collocano i celebri esperimenti sul ponte doppio (double bridge experiment) condotti da Deneubourg et al. alla fine degli anni '80. In questi studi, veniva collegato un nido di formiche a una fonte di cibo mediante due ponti artificiali di lunghezza diversa. Variando il rapporto tra le lunghezze dei due rami (definito $r = \frac{l_l}{l_s}$), si osservava come le formiche, inizialmente distribuite casualmente tra i due cammini, finivano progressivamente per convergere verso il ramo più corto.

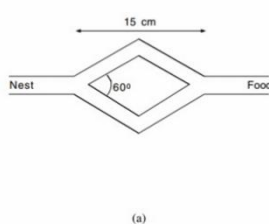


Figure 1.1
Experimental setup for the double bridge experiment. (a) Branches have equal length. (b) Branches have different length. Modified from Goss et al. (1989).

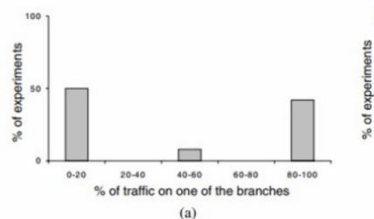


Figure 1.2
Results obtained with *Iridomyrmex humilis* ants in the double bridge experiment. (a) Results for the case in which the two branches have the same length ($r = 1$); in this case the ants use one branch or the other in approximately the same number of trials. (b) Results for the case in which one branch is twice as long as the other ($r = 2$); here in all the trials the great majority of ants chose the short branch. Modified from Goss et al. (1989).

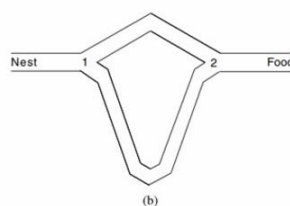


Figure 1.1
Experimental setup for the double bridge experiment. (a) Branches have equal length. (b) Branches have different length. Modified from Goss et al. (1989).

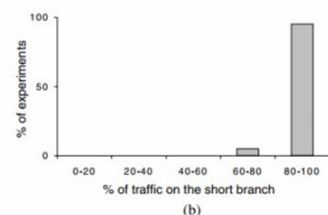


Figure 1.2
Results obtained with *Iridomyrmex humilis* ants in the double bridge experiment. (a) Results for the case in which the two branches have the same length ($r = 1$); in this case the ants use one branch or the other in approximately the same number of trials. (b) Results for the case in which one branch is twice as long as the other ($r = 2$); here in all the trials the great majority of ants chose the short branch. Modified from Goss et al. (1989).

Tuttavia, l'esperimento mostra anche un limite: se la colonia converge inizialmente su un **percorso subottimale**, l'**evaporazione del feromone** potrebbe non essere sufficientemente veloce da permettere l'abbandono di tale scelta in favore di percorsi migliori.

Ciononostante, il valore di questi esperimenti risiede nell'evidenza che il **comportamento collettivo** delle formiche è intrinsecamente **ottimizzante**. La capacità di convergere verso percorsi minimi tra due punti dello spazio, tramite l'utilizzo di regole probabilistiche fondate su informazione esclusivamente locale, fornisce la base teorica per la costruzione degli **artificial ants**.

Questi ultimi vengono modellati come **agenti computazionali** che si muovono su un grafo, simulando l'ambiente fisico in cui le formiche si muovono nella realtà. Attraverso l'impiego di **pheromone-based heuristics**, tali agenti sono in grado di esplorare lo spazio delle soluzioni e di trovare, progressivamente, il percorso più breve tra due nodi – analogamente a quanto accade tra il nido e la fonte di cibo. Questo approccio ha portato allo sviluppo della metaeuristica **Ant Colony Optimization (ACO)**, oggi ampiamente utilizzata per affrontare problemi di ottimizzazione combinatoria in vari domini applicativi.

Problemi di Ottimizzazione Combinatoria

I **problemi di ottimizzazione combinatoria** rappresentano una classe fondamentale nell'ambito dell'intelligenza artificiale, della teoria dei grafi e dell'ottimizzazione discreta. In termini generali, un problema di ottimizzazione combinatoria richiede di determinare i valori di una serie di **variabili discrete** in modo tale da ottimizzare (massimizzare o minimizzare) una specifica **funzione obiettivo (fitness function)**.

Formalmente, un problema di questo tipo può essere descritto da una quadrupla $(\mathcal{I}, f, m, g)$, dove:

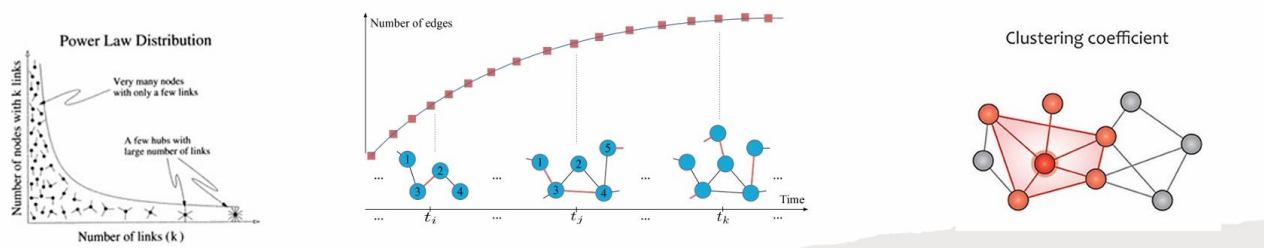
- $\mathcal{I}$ è l'insieme delle **istanze del problema**, ognuna delle quali rappresenta una configurazione completa dei parametri che definiscono un particolare caso del problema generale;
- $f(x)$ è l'insieme delle **soluzioni ammissibili** per una data istanza $x \in \mathcal{I}$;
- $m(x, y)$ rappresenta una **misura numerica** associata alla soluzione $y \in f(x)$, che può indicare un costo, una distanza, una penalità o qualsiasi altro valore significativo da ottimizzare;
- $g(x, y)$ è la **funzione da ottimizzare**, e l'obiettivo è quindi trovare $y^* \in f(x)$ tale che $m(x, y^*) = g(x, y^*)$ risulti ottimale.

Numerosi problemi classici in informatica appartengono a questa categoria, come il **Shortest Path Problem**, il **Network Flow**, i problemi di **Trasporto**, **Matching**, **Assegnamento**, **Routing**, **Location**, e molti altri. Questi problemi sono spesso modellati tramite strutture grafiche, dove il sistema da ottimizzare è rappresentato da un **grafo**.

Nel contesto dei sistemi reali, tali reti presentano caratteristiche **topologiche non banali**, quali la **distribuzione secondo legge di potenza dei gradi (power-law)**, un elevato **clustering coefficient**, e una **dinamicità** temporale che rende la struttura mutevole nel tempo. A questo proposito si parla di **reti dinamiche**, definite da un grafo i cui nodi e archi sono annotati con istanti di apparizione e scomparsa. Questo modello consente di rappresentare in modo preciso la natura temporale di connessioni instabili, quali quelle nei sistemi di telecomunicazione, nei social network o nelle reti logistiche.

I **metodi esatti** per la risoluzione di problemi di ottimizzazione combinatoria garantiscono, in linea teorica, la determinazione della soluzione ottima. Tuttavia, essi presentano **costi computazionali proibitivi** quando vengono applicati a reti reali di grandi dimensioni o a istanze caratterizzate da **incertezza**, **dinamismo** e **informazione incompleta**. Per questo motivo, l'adozione di **algoritmi approssimati** si è rivelata essenziale nei contesti applicativi.

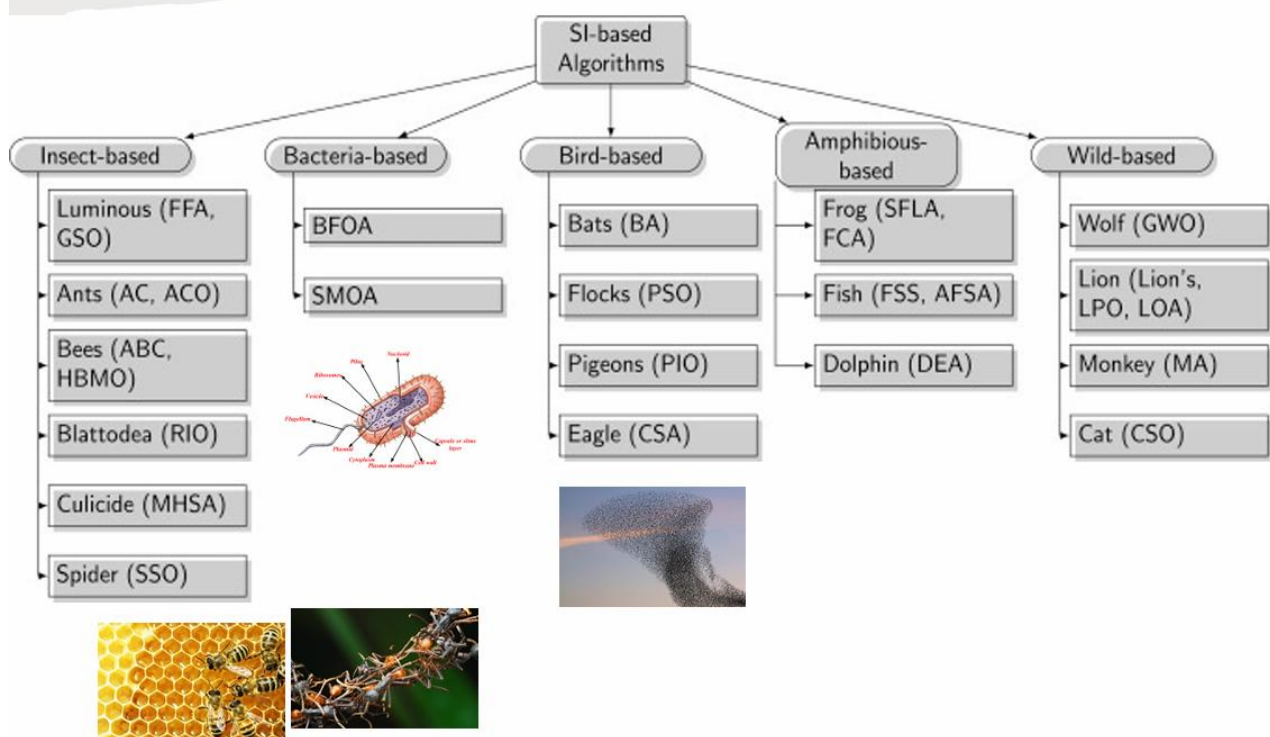
Tra questi, gli **algoritmi bio-ispirati** — come quelli derivati dalla teoria dell'evoluzione darwiniana, dal comportamento degli insetti sociali o dall'organizzazione collettiva degli stormi — offrono una soluzione flessibile e robusta. Gli algoritmi approssimati non dipendono dalla specifica struttura del problema e possono essere trattati come **black-box solvers**; non essendo greedy, esplorano un ampio spettro dello spazio delle soluzioni e si adattano naturalmente a problemi complessi, multimodali e non stazionari.



Algoritmi di Swarm Intelligence

La **Swarm Intelligence** è un paradigma ispirato al comportamento collettivo di sistemi **decentralizzati** e **auto-organizzati**, come colonie di formiche o stormi di uccelli. Gli individui, pur essendo semplici, interagiscono localmente senza coordinamento centrale, dando origine a un'intelligenza emergente che consente al gruppo di risolvere problemi complessi che il singolo non potrebbe affrontare.

Gli agenti operano con **informazioni locali**, **memoria limitata** e **azioni semplici**, ma il sistema è robusto: anche la perdita di alcuni elementi non compromette la performance complessiva. Questo principio è alla base di algoritmi come **Ant Colony Optimization** e **Particle Swarm Optimization**, ampiamente usati in problemi di ottimizzazione combinatoria e continua.



Ant Colony Algorithms

All'interno della metaeuristica **Ant Colony Optimization (ACO)**, sono state proposte numerose varianti, tra cui le tre più rilevanti:

- **Ant System (AS)**, introdotto da **Marco Dorigo** nel 1992, è il modello base in cui tutte le formiche aggiornano i feromoni in base alla qualità del percorso.
- **Min-Max Ant System (MMAS)**, sviluppato da **Hoos e Stützle** nel 1996, introduce limiti minimi e massimi ai livelli di feromone e consente l'aggiornamento solo alla formica migliore, migliorando la stabilità e prevenendo la stagnazione.
- **Ant Colony System (ACS)**, proposto da **Gambardella e Dorigo** nel 1997, bilancia *exploration* ed *exploitation* tramite una regola pseudo-casuale di transizione e introduce un aggiornamento locale dei feromoni per diversificare la ricerca.

Ant System

Nel **modello Ant System (AS)** la costruzione e il raffinamento progressivo delle soluzioni si articolano attorno a tre meccanismi cardine, ciascuno dei quali riproduce in forma astratta un aspetto fondamentale del *foraging behaviour* delle formiche reali.

La **Proportional Transition Rule** regola la scelta del successivo nodo da visitare durante la fase di **solution construction**. Per ogni formica posizionata in i , la probabilità di selezionare un nodo adiacente j è definita da:

$$p_{ij}^k(t) = \begin{cases} \frac{\tau_{ij}(t)^\alpha \cdot \eta_{ij}^\beta}{\sum_{l \in J_i^k} \tau_{il}(t)^\alpha \cdot \eta_{il}^\beta} & \text{if } j \in J_i^k \\ 0 & \text{if } j \notin J_i^k \end{cases}$$

dove $\tau_{ij}(t)$ rappresenta l'intensità del **pheromone trail** sull'arco (i,j) , mentre η_{ij} (o un'altra metrica di **visibility**) quantifica l'appetibilità euristica del nodo. I parametri α e β modulano rispettivamente l'importanza relativa del **feromone** e **dell'informazione euristica**: valori elevati di α inducono un comportamento spiccatamente imitativo (*exploitation*), mentre un β più alto intensifica il peso della distanza o del costo locale, favorendo l'*exploration* di percorsi poco battuti. L'equilibrio fra questi due contributi determina la capacità del sistema di evitare tanto la convergenza prematura quanto la dispersione eccessiva dello *search effort*.

Una volta completato il tour, interviene la **Reinforcement Rule**. L'agente deposita lungo ogni arco (i,j) attraversato una quantità di feromone inversamente proporzionale alla **Tour Length** L^k :

$$\Delta\tau_{ij}^k(t) = \begin{cases} \frac{Q}{L^k(t)} & \text{if } (i,j) \in T^k(t) \\ 0 & \text{if } (i,j) \notin T^k(t) \end{cases}$$

dove Q è un coefficiente di scala che può essere interpretato come *learning rate* globale. L'azione di rinforzo implementa il meccanismo di **positive feedback**: i cammini più brevi ottengono un incremento di feromone maggiore, accrescendo la probabilità di essere scelti nelle iterazioni successive e riducendo progressivamente la *Loss* associata alla soluzione corrente.

Il terzo meccanismo, la **Global Updating Rule**, introduce un processo di **pheromone evaporation** che simula il degrado chimico osservato in natura e, sul piano algoritmico, svolge la funzione di **regularization**. Al termine di ciascuna iterazione, i livelli di feromone vengono aggiornati secondo:

$$\tau_{ij}(t + 1) = (1 - \rho) \tau_{ij}(t) + \sum_{k=1}^m \Delta\tau_{ij}^k(t)$$

con $0 < \rho < 1$ parametro di evaporazione e m numero di formiche. Riducendo il valore di τ_{ij} a ogni passo, l'algoritmo previene l'eccessivo *over-reinforcement* di percorsi subottimali e mantiene attivo un grado minimo di *exploration*. Un ρ troppo alto, tuttavia, può cancellare prematuramente informazioni utili.

Min-Max Ant System

Il Min-Max Ant System è una variante dell'algoritmo Ant System in cui solo la formica con il miglior percorso, globale o dell'iterazione, può depositare feromone sugli archi. Tutti gli archi partono con il valore massimo di feromone τ_{max} per favorire l'esplorazione iniziale, e la quantità di feromone su ogni arco è limitata tra τ_{min} e τ_{max} per evitare stagnazione. L'aggiornamento del feromone combina evaporazione e deposito solo sul miglior percorso, bilanciando così esplorazione e sfruttamento per migliorare la ricerca della soluzione ottimale.

Reinforcement Rule

La quantità di feromone $\Delta\tau_t(i, j)$ depositata su un arco (i, j) al tempo t è pari a $\frac{Q}{L(t)}$ se l'arco fa parte del miglior percorso $T(t)$ (che può essere il miglior tour dell'iterazione o il miglior globale). Altrimenti, la quantità depositata è zero. Qui Q è una costante e $L(t)$ è la lunghezza del miglior tour considerato.

$$\Delta\tau_{ij}^{best}(t) = \begin{cases} \frac{Q}{L^{best}(t)} & \text{if } (i, j) \in T^k(t) \\ 0 & \text{if } (i, j) \notin T^k(t) \end{cases}$$

Global Updating Rule:

Il valore di feromone su ogni arco viene aggiornato secondo la formula:

$$\tau_{ij}(t + 1) = (1 - \rho) \tau_{ij}(t) + \rho \cdot \Delta\tau_{ij}^{best}(t)$$

dove ρ è il tasso di evaporazione. Solo la formica che ha trovato il miglior tour è autorizzata a modificare i valori di feromone con il suo deposito.

Ant Colony System

L'Ant Colony System è un'evoluzione dell'algoritmo Ant System che cerca di bilanciare meglio **esplorazione** ed **esplorazione**. La regola di transizione proporzionale diventa la **Pseudorandom Proportional Rule**, in cui la scelta del prossimo nodo da visitare dipende da un valore casuale q uniformemente distribuito in $[0,1]$ e da una soglia q_0 . Se $q \leq q_0$, la formica sceglie il nodo che massimizza il prodotto tra la quantità di feromone e una misura euristica di qualità (ad esempio l'inverso della distanza). Altrimenti, si applica la regola classica di Ant System, scegliendo il nodo con probabilità proporzionale a questo prodotto. Questo meccanismo favorisce **l'esplorazione** delle informazioni di feromone quando $q \leq q_0$ e **l'esplorazione** altrimenti.

$$j = \begin{cases} \operatorname{argmax}_{j \in J_i^k} \{ \tau_{ij} [\eta_{ij}]^\beta \} & \text{if } q \leq q_0 \\ \text{AS} & \text{otherwise} \end{cases}$$

Una differenza importante rispetto ad Ant System è l'introduzione della **local updating rule**. Durante la costruzione passo dopo passo della soluzione, ogni volta che una formica attraversa un arco, diminuisce la quantità di feromone su quell'arco secondo la formula:

$$\tau_{ij}(t + 1) = (1 - \varphi) \tau_{ij}(t) + \varphi \cdot \tau_{ij}(0)$$

dove φ è un parametro di aggiornamento locale e τ_0 è il valore iniziale di feromone. Questo serve a evitare che un arco venga troppo rinforzato troppo presto, favorendo così l'esplorazione.

L'**aggiornamento globale** invece avviene solo dopo che tutte le formiche hanno costruito la loro soluzione, e viene applicato unicamente dalla formica che ha trovato il miglior percorso assoluto. Questo aggiornamento segue la regola:

$$\tau_{ij}(t + 1) = (1 - \rho) \tau_{ij}(t) + \rho \cdot \Delta\tau_{ij}^{best}(t)$$

dove ρ è il tasso di evaporazione.

Aspetto	Local Updating Rule	Global Updating Rule
Quando si applica	Durante la costruzione della soluzione	Al termine di ogni iterazione
Chi aggiorna	Ogni formica	Solo la formica migliore
Effetto	Favorisce l'esplorazione	Favorisce l'exploitation
Scopo Principale	Esplorazione/Diversificazione	Exploitation/Convergenza

Algoritmo ACO

```

Inizio

  Inizializza i parametri

  Mentre il criterio di arresto non è soddisfatto fare

    Posiziona ogni formica su un nodo iniziale

    Ripeti

      Per ogni formica fare

        Scegli il prossimo nodo applicando la regola di transizione di stato

        Applica l'aggiornamento del feromone passo dopo passo (aggiornamento locale)

      Fine per

    Finché ogni formica non ha costruito una soluzione

    Aggiorna la miglior soluzione trovata

    Applica l'aggiornamento offline del feromone (aggiornamento globale)

  Fine mentre

Fine
  
```


Vantaggi e Svantaggi

L'Ant Colony System è un metaeuristica basata su meccanismi di **feedback positivo**, che permettono una rapida individuazione di buone soluzioni. Questo approccio risulta particolarmente efficace per problemi combinatori come il **Travelling Salesman Problem (TSP)**, e può essere impiegato anche in contesti dinamici, dove ad esempio le distanze o le condizioni della rete possono cambiare nel tempo. La capacità dell'algoritmo di adattarsi lo rende utile in applicazioni reali che richiedono flessibilità.

Tuttavia, l'ACS presenta anche alcune **criticità**. Poiché si basa su distribuzioni di probabilità che si modificano a ogni iterazione, l'algoritmo può produrre soluzioni molto diverse se i parametri **non sono ben calibrati**. Inoltre, la metodologia è prevalentemente di tipo sperimentale più che teorico, il che **rende difficile prevedere con precisione il comportamento in ogni scenario**. Anche se la convergenza è garantita, i tempi necessari non sono sempre noti o facilmente controllabili.

Performance

Dal punto di vista delle performance, **l'ACS ha dimostrato di ottenere le migliori soluzioni su problemi di piccole dimensioni** (come il TSP con 30 città). Su problemi più ampi, sebbene non sempre riesca a trovare la soluzione ottima, è spesso in grado di convergere **verso soluzioni di buona qualità in tempi accettabili**. Tuttavia, su problemi statici e ben strutturati, gli algoritmi esatti o specialistici risultano ancora difficili da superare.

Tra le principali applicazioni pratiche dell'Ant Colony System troviamo:

- il problema del commesso viaggiatore (TSP)
- lo scheduling
- la colorazione dei grafi
- e altri problemi di ottimizzazione combinatoria